

A Formal Framework for Reactive Heterogeneous Multirobot Task Allocation in Uncertain Semantic Environments

Lin Li[✉], *Graduate Student Member, IEEE*, Zhangli Zhou[✉], Ziyang Chen[✉], Hao Wang[✉],
Zhen Kan[✉], *Senior Member, IEEE*, and Jiahu Qin[✉], *Senior Member, IEEE*

Abstract—Existing multirobot task allocation (MRTA) research primarily assumes an environment with known geometric and semantic information. However, in most real-world scenarios, semantic information, such as the locations and classifications of landmarks, is often uncertain. This uncertainty is exacerbated by dynamic requirements, like the sudden addition or removal of tasks, making it challenging for robots to respond reactively. Moreover, tasks in MRTA typically involve multiple constraints, including temporal requirements, diverse capabilities, varying resource needs, and inter-task dependencies. To address these challenges, we propose a reactive task allocation framework for heterogeneous multirobot systems that accounts for temporal requirements and multiple task constraints described by LTL^R. Our approach assumes an environment with known geometry but unknown semantic landmarks. To efficiently solve task allocations, we encode LTL^R along with the system's states into a proposed planning decision tree for exploration. Upon detecting a relevant semantic landmark, the reactive multiconstraint planning decision tree (RMC-PDT) is triggered for re-planning. Extensive experiments validate three key features of our method: 1) efficient reactive planning; 2) multiconstraint task solving; and 3) scalability.

Index Terms—Heterogeneous multirobot systems, linear temporal logic (LTL), reactive planning, task allocation.

I. INTRODUCTION

MULTIROBOT systems have been widely deployed in mission-critical applications, such as search and rescue [1], intelligent transportation [2], material transportation [3], and disaster response [4]. Within this context, multirobot task allocation (MRTA) serves as a critical component in enabling effective coordination within such systems. Existing MRTA methods [5], [6], [7] have primarily focused on relatively straightforward tasks, such as assigning robots to predefined locations while avoiding obstacles [8], [9], [10]. However, these efforts often fall short when confronted with more complex multirobot tasks. By logically reasoning about the

temporal ordering of events at the task level, temporal logic enables the specification of richer behaviors beyond traditional go-to-goal navigation [11], [12]. For example, the work of [13] specified the number of robots required to satisfy the collaborative task (e.g., two robots visit region A, three robots eventually visit regions B and C repeatedly, while always avoiding collision with obstacles). The work of [14] further considered different capabilities of the robots required for the task, and a decentralized framework for heterogeneous redundant systems was introduced in [15]. Moreover, tasks with divisible and indivisible resource constraints were addressed in [16], while Duan et al. [17] proposed a potential game framework to handle heterogeneous task allocation under resource depletion. In addition, energy-aware MRTA with battery capacity constraints was considered in application-driven settings [18]. Despite recent progress, multirobot tasks are often subject to multiple constraints, such as temporal requirements, resource constraints, and relationships between tasks. In addition to multiconstraint task specifications, uncertain semantic environments introduce another critical challenge. Certain landmarks may carry semantic information that directly affects task execution, such as access restrictions triggered by detected objects. Motivated by these practical challenges, this work aims to develop a reactive MRTA framework that addresses multiple constraints in uncertain semantic environments.

Linear temporal logic (LTL), as a formal language, offers rich expressiveness for describing collaborative tasks with temporal constraints in multirobot systems [19], [20], [21]. However, as task complexity increases, using traditional temporal logic to describe them becomes cumbersome and computationally intensive [22], particularly for large robot teams. To address this, several studies have proposed new forms of temporal logic to better handle a variety of challenging tasks. For instance, in [13], LTL was extended to cLTL and cLTL+ to represent tasks with counting constraints in homogeneous robotic systems. Similarly, CensusSTL [23] was capable of expressing task requirements involving specific quantities of robots. To further improve expressiveness, a hierarchical temporal logic was proposed to transform “flat” temporal logic into a hierarchical format in [24]. In addition, distributed synthesis approaches based on task decomposition have been developed to efficiently handle finite-horizon temporal logic tasks [25]. Due to the specificity of the logical structure, it is difficult to encode other factors, such as heterogeneity, dynamic resources, or task dependencies, within the above framework. To characterize such tasks, capability temporal

Received 16 August 2025; revised 25 December 2025 and 11 February 2026; accepted 22 February 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U25A20473. This article was recommended by Associate Editor C. Shen. (*Corresponding author: Zhangli Zhou.*)

The authors are with the Department of Automation, University of Science and Technology of China, Hefei, Anhui 230026, China (e-mail: zhangli@ustc.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCYB.2026.3668425>.

Digital Object Identifier 10.1109/TCYB.2026.3668425

logic (CaTL) [14] and mcSTL [16] primarily focused on fixed capabilities or resources, without considering dynamic or varying conditions. Moreover, the above works cannot capture relationships between tasks. Our previous work [26], [27] considered not only temporal requirements, but also the relationships between tasks. However, existing approaches mainly handle these constraints in isolation, limiting their expressiveness and adaptability in complex, heterogeneous scenarios. Motivated by this challenge, we propose $\text{LTL}^{\mathcal{R}}$, which features two layers of logic for enhanced interpretability and expressiveness. The inner logic employs LTL to represent individual robot behaviors, while the outer logic encompasses group tasks with multiple constraints, including subtasks, variable resources, and inter-task constraints. Robots are categorized by their capabilities into nonstorage and storage. Nonstorage capabilities are inherent and fixed. In contrast, storage capabilities allow the accumulation of resources that may vary over time or across tasks. Inter-task constraints are classified as unrelated (no dependencies), compatible (tasks with shared requirements that can use the same robot group), and exclusive (tasks that cannot share robots).

Besides, approaches, such as cLTL and CaTL, model each robot as an individual transition system. This modeling paradigm leads to limited scalability and reduced computational efficiency in large-scale scenarios. In particular, CaTL encodes both temporal and capability constraints into a mixed-integer linear programming (MILP) formulation. As the number of capability combinations, task regions, and robots increases, the dimensionality of the MILP formulation grows rapidly, severely limiting the scalability and practical applicability. Sampling-based methods [28], [29] can avoid exhaustive enumeration and thus improve computational efficiency. However, such methods may suffer from inefficient exploration, leading to instability and potential inefficiency in the planning process. Furthermore, advanced optimization strategies, such as dynamic resource allocation [30] and iterative clustering [31], have been introduced to enhance computational efficiency by optimizing resource usage or partitioning global problems. Nevertheless, these approaches typically focus on utility maximization. They lack the formal capability to model high-level temporal logic and inter-task constraints required for complex missions. To address these issues, we propose a tree-search-based task allocation and planning method that constructs a decision tree based on feasible transitions in the Büchi automaton. At each node, task allocations are explored by solving an ILP subproblem under the current state constraints. Adaptive strategies are then applied to handle resource variability and inter-task constraints. Our method adopts the “local planning-global integration” decoupling mechanism, which decomposes the overall task allocation and planning process into smaller, automaton-driven subproblems. This design significantly reduces the dimensionality of global variables to improve computational efficiency.

Furthermore, many existing works assume known deterministic environments, yet most real-world environments that robots operate in are uncertain [32]. Such environments were considered in [33] and [34] with unknown geometry and known regions of interest for LTL tasks. Instead, [35] considered environments with known geometry but uncertain semantic structures, where the labels and positions of regions of interest are uncertain. Besides, Li et al. [36]

TABLE I
COMPARISON OF KEY FEATURES OF REPRESENTATIVE
MRTA FRAMEWORKS

Methods	Heterogeneous	Semantic Information	Uncertain Tasks	Task Constraints	Reactive Planning
cLTL [13]	×	×	×	×	×
CaTL [14]	✓	×	×	×	△
Var-CICS [27]	✓	×	×	✓	×
IPDTS [39]	✓	×	×	×	✓
Perception-based method [35]	×	✓	×	×	✓
Ours	✓	✓	✓	✓	✓

further explored environment-driven autonomy by leveraging LLMs to generate and verify LTL task specifications in a closed loop, but did not target reactive task reconstruction under semantic uncertainty. In this work, the environment is assumed to have known geometry and task regions. However, the positions and semantic information of certain specific landmarks remain unknown, which determines the potential relevance of these landmarks to task execution. Unlike [35], where replanning frequency heavily depends on the accuracy of the initial semantic maps, our approach triggers replanning specifically upon landmark detection, ensuring independence from initial map accuracy. Instead of constructing the product automaton, we propose an innovative automaton-reconstruction-based search method that dynamically detects landmarks to trigger the addition or removal of temporary tasks. In response to such changes, a new Büchi automaton is constructed based on the updated task specification. The planner then explores the next reachable automaton states from the current state. This strategy significantly reduces the complexity of the state space. Moreover, most existing reactive planning methods focus on path-level adjustments without modifying the underlying task logic structure or constraints [37], [38]. In contrast, our approach addresses the task uncertainty by detecting landmarks with unknown semantics. The associated task requirements are then dynamically updated, enabling reactive planning at the task level. In addition, unlike the coordination between global and local tasks in [39], our method supports dynamic reconstruction of the task logic along with the corresponding automaton. This design achieves greater flexibility and expressive power in handling semantic uncertainty and temporary task requirements.

Based on the above analysis, to comprehensively highlight the gaps in existing research and position our contributions, Table I summarizes the key differences between our proposed method and representative methods. The considered features include robot heterogeneity, semantic information, the ability to handle uncertain task requirements driven by semantic variability, multiconstraint problem-solving capability, and reactive planning. These features represent the core challenges faced by multirobot systems in complex environments. As shown in Table I, most existing approaches cover only a subset of these features and thus struggle to meet the comprehensive demands of complex task scenarios.

In this work, we propose a formal reactive task allocation framework for heterogeneous multirobot systems operating in uncertain environments, which addresses tasks with temporal constraints and multiple task-level requirements. These multiconstraint tasks are specified using $\text{LTL}^{\mathcal{R}}$, which is

encoded in the planning decision tree [40] with a more concise representation to facilitate efficient exploration of system states and tasks. To handle multiple constraints, we propose a reactive multiconstraint planning decision tree (RMC-PDT). RMC-PDT adapts different inter-task constraints to specific search patterns and accommodates various resource types. The associated resources are dynamically updated in response to temporal changes and task demands. When the semantic information and positions of detected landmarks satisfy predefined triggers, such as task addition or removal, an automaton-reconstruction-based search procedure is initiated as part of reactive planning. This procedure updates the task-level logic and associated requirements. A new Büchi automaton is then reconstructed accordingly. Based on the current system state and the reconstructed automaton, the planner explores the subsequent reachable states. For each reachable state, we search for a task allocation that satisfies the multiconstraint requirements, which in turn drives the next round of the RMC-PDT exploration. Such an approach does not require considering all dependencies between tasks, facilitating a real-time solution. The generated task plan is subsequently forwarded to the motion planning module, enabling robots to execute tasks in alignment with the generated plan. This “local planning and global integration” decoupling mechanism incrementally decomposes the task execution flow into subproblems aligned with automaton state transitions. Consequently, it significantly reduces computational complexity while ensuring both completeness and validity.

Our main contributions are summarized as follows.

- 1) We introduce a two-layer logic, $LTL^{\mathcal{R}}$, to represent temporal requirements alongside resource and inter-task constraints, which offers enhanced expressiveness with a more structured and transparent formulation.
- 2) We propose an automaton-reconstruction search method that dynamically updates the task automaton upon detecting semantic changes, which enables the system to perceive, allocate, and plan tasks to meet emerging requirements.
- 3) We develop a formal reactive task allocation and planning framework under uncertain semantic environments. Unlike traditional product-automaton approaches, a planning-decision-tree-based method (RMC-PDT) is adopted to explore allocations satisfying both $LTL^{\mathcal{R}}$ specification and temporary task requirements.
- 4) Theoretical analysis proves the completeness and validity of our approach. Extensive experiments validate three key features: 1) efficient reactive planning; 2) robust handling of multiconstraint tasks; and 3) scalability. Experimental results demonstrate that our method can tackle up to 3×10^4 robots within approximately 1 min and resolve over 100 automaton states in around 1 s.

Notations: Let $\mathbb{N}$ and $[N]$ denote the set of natural numbers and the shorthand notation for $\{1, \dots, N\}$, respectively. Given a set A , let $|A|$ and 2^A denote the cardinality and power set of A , respectively. Given two sets A and B , $A \setminus B = \{x | x \in A, x \notin B\}$ denotes the set difference.

II. PROBLEM FORMULATION

A. Environment and Multirobot System

The uncertain semantic environment is denoted as $Env = (M, L, AP, Q, \mathcal{L}_M, \mathcal{L}_a)$, where M is a map containing $m \in \mathbb{N}$

nonoverlapping task regions $M_i, i \in [m]$, and the nontask region $\bar{M}$, and $L = \{l_1, l_2, \dots, l_n\}$ is the set of $n \in \mathbb{N}$ landmarks on M with uncertain semantics. Each landmark is in the form of $l_i = (p_i^l, c_i)$, $i \in [n]$, with $p_i^l = (x, y) \in M$ and $c_i \in C_L$ indicating the position and category, respectively, where C_L represents the finite set of landmark classes (e.g., person, trash, door, etc.). The positions and semantics of landmarks are initially uncertain but can be detected via methods such as object detection and SLAM. In this work, the perceptual modules (e.g., RGB-D cameras) are used to detect the landmarks, and a CNN-based object detector (YOLO) [41], [42] is used to infer their semantic information. AP is the set of atomic propositions and Q denotes the finite set of task states (i.e., abstract forms of subtasks serving as a bridge to facilitate the mapping of relationships between tasks), $\mathcal{L}_M: Q \rightarrow M$ and $\mathcal{L}_a: Q \rightarrow AP$ map the task states to M and AP , respectively, where $\mathcal{L}_M(q) = p$ and $\mathcal{L}_a(q) = \pi$ with $q \in Q$, $p = (x, y) \in M$ and $\pi \in AP$.

Consider a set of N heterogeneous robots $A = \{a_n\}_{n=1}^N$ operating in the environment Env . Each robot is denoted as a tuple $a_n = (p_0, p, CAP, \tilde{R})$, where p_0 and p are the initial and current positions of a_n , respectively. The robot capabilities $CAP = \{Cap_j\}_{j \in [N_C]} \cup \{o\}$ contain a set of N_C different capabilities [such as the loading capability (LC)] required for the task and the perception capability o for semantic detection. Specifically, we consider two different types of capabilities: storage capabilities and nonstorage capabilities. Storage capabilities encompass a robot's capacity to contain and transport specific resources, such as disinfectants or active reagents, within a hospital setting. Conversely, nonstorage capabilities are intrinsic to the robot itself and include, for example, the ability to load medicines onto a shelf. These capabilities can be represented as $Cap_j = (n_j^{\mathcal{R}}, c_j)$ or $Cap_j = (n_j^{\mathcal{C}}, c_j)$, where the superscript $\mathcal{R}$ and $\mathcal{C}$ indicate the storage and the nonstorage capability, respectively, and c_j is the maximum capacity of resources the robot can transport or load. The perception capability $o = 1$ indicates that the robot is equipped with a camera to detect objects, and $o = 0$ otherwise. Let $\tilde{R} = \{(n_j^X, v_j)\}_{j \in [N_C], X \in \{\mathcal{R}, \mathcal{C}\}}$ denote the current resource and its corresponding volume possessed by the robot. For storage capabilities, i.e., $X = \mathcal{R}$, we have $0 \leq v_j \leq c_j$, since the amount of resources can vary with respect to time (time-dependent resource) or task operation (task-dependent resource). The resource value can typically be formulated as a take or time-dependent function f , without exceeding its corresponding capacity. For nonstorage capabilities, i.e., $X = \mathcal{C}$, we have $v_j = c_j$, as such capabilities are invariant.

Example 1: Consider a hospital with uncertain semantic landmarks, where objects such as “person” and “trash” may be associated with local tasks. Each robot is equipped with one or more of the following capabilities.

- 1) LC, a nonstorage capability for lifting or placing medicines.
- 2) Disinfection capability (DC), a storage capability for transporting and spraying disinfectant.
- 3) Virus-detection capability (VC), a storage capability for storing and using reagents for virus detection.

These resources may vary over time or with specific tasks. In particular, the disinfectant is a task-dependent resource, with its volume decreasing during the disinfection operation. The reagent is both task-dependent and time-dependent, as its effectiveness and volume diminish over time and with the use

in virus-detection tasks. Also, these resources can be refilled by supplementary operations.

Remark 1: Beyond the hospital scenario, similar multi-factor combined tasks are also common in various applications such as disaster response, warehouse logistics, and port operations. These scenarios typically involve time-dependent resources (e.g., battery power, oxygen cylinders), task-dependent resources (e.g., disposable medical supplies, cold-chain refrigerants), inherent capabilities (e.g., robotic arm handling, debris removal), as well as various inter-task constraints such as compatibility and exclusivity. Therefore, developing a formal framework capable of handling multifactor coupling and dynamic task adjustments in MRTA is of broad applicability and practical significance.

B. Two-Layer LTL

As a formal language, LTL is widely used for specifying complex tasks, which are defined over AP as [43]

$$\varphi ::= \text{true} \mid \pi \mid \varphi_1 \wedge \varphi_2 \mid \neg \varphi \mid \bigcirc \varphi \mid \varphi_1 \cup \varphi_2$$

where true is the boolean value, $\pi \in AP$ represents an atomic proposition, $\neg$ (negation) and $\wedge$ (conjunction) are Boolean operators, φ_1 and φ_2 are LTL formulas, and $\bigcirc$ (next) and $\cup$ (until) are temporal operators. Other propositional logic operators such as $\vee$ (disjunction), $\rightarrow$ (implication), and temporal operators such as $\Box$ (always) and $\Diamond$ (eventually) can be found in [43].

In this work, we extend LTL to a two-layer temporal logic $\text{LTL}^{\mathcal{R}}$, with an inner logic, LTL, for specifying tasks of the individual robot and the outer logic for group tasks. An LTL formula can be translated to a nondeterministic Büchi Automaton (NBA) $B = \{S, s_0, \Sigma, \delta, \mathcal{F}\}$ [44], where S is a finite set of states; $s_0 \in S$ is the initial state; $\Sigma = 2^{AP}$ is the alphabet; $\delta \subseteq S \times S \rightarrow 2^\Sigma$ is the transition function, and $\mathcal{F} \subseteq S$ is the set of accepting states. The word $\sigma = \sigma_1 \sigma_2 \sigma_3, \dots$ is an infinite sequence, where $\sigma_i \in \Sigma$. Let $s = s_0 s_1 s_2, \dots$ be a run of the NBA induced by σ , where $\sigma_{i+1} \in \delta(s_i, s_{i+1})$, $\forall i \in \mathbb{Z}_{\geq 0}$. If the infinite sequence σ can generate at least one run s that intersects the accepting states $\mathcal{F}$ infinitely many times, B is said to accept σ . Let σ_{a_n} be the trajectory of robot a_n and the trajectory at step t is denoted as $\sigma_{a_n}^t$. We denote by $\sigma_{a_n}^t \models \pi$ if and only if $\pi \in \sigma_{a_n}^t$, where $\models$ is the satisfaction relation.

The outer logic of $\text{LTL}^{\mathcal{R}}$ specifies the group behaviors of the multirobot system by an induced task proposition $\varphi^{R,B} \in \mathcal{TP}$, where $\mathcal{TP}$ is the set of task propositions, φ is an inner logic formula introduced before, $R = \{(n_j^X, v_j)\}_{j \in [N_C], X \in \{\mathcal{R}, \mathcal{C}\}}$ is the set of resources and their amount needed for the task φ , and $B \in \mathbb{Z}$ is the task batch indicating compatible, exclusive, and unrelated inter-task constraints. Specifically, the compatible tasks indicate a set of tasks of the same types and requirements so that the same group of robots can be reassigned for other compatible tasks. Exclusive tasks indicate that the robots that participated in one task cannot be reassigned for its corresponding exclusive tasks. Unrelated tasks, on the other hand, impose no constraints on the robot task allocation (i.e., any robots can be assigned to unrelated tasks). The proposition $\varphi^{R,B}$ is evaluated as true if the inner logic formula φ is satisfied by a group of robots with the desired resource and the task batch requirement. Due to the consideration of $\varphi^{R,B}$, the uncertain semantic map Env is modified as

TABLE II
REQUIRED RESOURCES FOR TASKS

tasks	LC/box	DC/L	VC/mg
φ_1	11	12	102
φ_2	12	10	101
φ_3	18	18	92
φ_4	14	16	100
φ_{new}	19	13	112

$Env = (M, L, AP, Q, \mathcal{L}_M, \mathcal{L}_a, \mathcal{L}_t)$, where $\mathcal{L}_t: Q \rightarrow \mathcal{TP}$ maps a task state to a task proposition $\varphi^{R,B} \in \mathcal{TP}$.

After defining the inter and outer logic, we define the syntax and semantics of $\text{LTL}^{\mathcal{R}}$. In particular, the team trajectories of robots is denoted as $\Sigma_A = \{\sigma_{a_1}, \sigma_{a_2}, \dots, \sigma_{a_N}\}$ and these collection of trajectories at step t is denoted as (Σ_A, t) .

Definition 1: The syntax of $\text{LTL}^{\mathcal{R}}$ is given as

$$\phi ::= \text{true} \mid \varphi^{R,B} \mid \phi_1 \wedge \phi_2 \mid \neg \phi \mid \bigcirc \phi \mid \phi_1 \cup \phi_2 \quad (1)$$

where $\varphi^{R,B}$ is a task proposition which can be true or false . ϕ , ϕ_1 , and ϕ_2 are $\text{LTL}^{\mathcal{R}}$ formulas. The descriptions of Boolean and temporal operators are the same as LTL.

Definition 2: The semantics of $\text{LTL}^{\mathcal{R}}$ are defined as follows:

- 1) $(\Sigma_A, t) \models \text{true}$;
- 2) $(\Sigma_A, t) \models \varphi^{R,B} \Leftrightarrow \left\{ \sum_{\sigma_{a_n}^t \models \varphi} a_n \cdot v_j \geq \varphi^{R,B} \cdot v_j \right\}_{j \in [N_C]}$ and $\{a_n \mid \sigma_{a_n}^t \models \varphi\}_{n \in [N]} \models B$;
- 3) $(\Sigma_A, t) \models \phi_1 \wedge \phi_2 \Leftrightarrow (\Sigma_A, t) \models \phi_1$ and $(\Sigma_A, t) \models \phi_2$;
- 4) $(\Sigma_A, t) \models \phi_1 \vee \phi_2 \Leftrightarrow (\Sigma_A, t) \models \phi_1$ or $(\Sigma_A, t) \models \phi_2$;
- 5) $(\Sigma_A, t) \models \neg \phi \Leftrightarrow (\Sigma_A, t) \not\models \phi$;
- 6) $(\Sigma_A, t) \models \bigcirc \phi \Leftrightarrow (\Sigma_A, t+1) \models \phi$;
- 7) $(\Sigma_A, t) \models \phi_1 \cup \phi_2 \Leftrightarrow \exists l \geq 0, (\Sigma_A, t+l) \models \phi_2, \forall 0 \leq l' < l, (\Sigma_A, t+l') \not\models \phi_1$.

$(\Sigma_A, 0) \models \phi$ implies Σ_A satisfies ϕ , which is abbreviated as $\Sigma_A \models \phi$. The satisfaction of $(\Sigma_A, t) \models \varphi^{R,B}$ requires the following two conditions: 1) the sum of the resources possessed by the robots satisfying φ needs to be no less than the required amount specified by R and 2) robots satisfying φ need to meet the inter-task constraints B .

To facilitate the subsequent formulation and analysis of the task allocation process, we define the following variables and notations. Specifically, the task allocation is denoted by $Z = [z_1, z_2, \dots, z_N]$, where $z_i = 1$ indicates the robot a_i is assigned with the task and vice versa. The plan is denoted as $\Pi = (s, q, Z)$, where $s = s_0 s_1, \dots, q = q_0 q_1, \dots$, and $Z = Z_1 Z_2, \dots$ are sequences of automaton states, task states, and task allocations, respectively.

C. Problem

The following example is introduced to illustrate the problem to be addressed in this work.

Example 2: Continuing with Example 1, the atomic proposition indicates the task to be performed, e.g., π_{ward} indicates the task of visiting the ward. The robots are required to complete the following tasks:

$$\begin{aligned} \varphi_1 &= \Box \Diamond \pi_{\text{ward}} \\ \varphi_2 &= \Box \Diamond \pi_{\text{central_laboratory}} \wedge \Box \Diamond \pi_{\text{disinfection_room}} \\ \varphi_3 &= \Box \Diamond \pi_{\text{waiting_room}} \wedge \Box \Diamond \pi_{X_ray} \\ \varphi_4 &= \Box \Diamond \pi_{\text{doctor's_room}} \end{aligned}$$

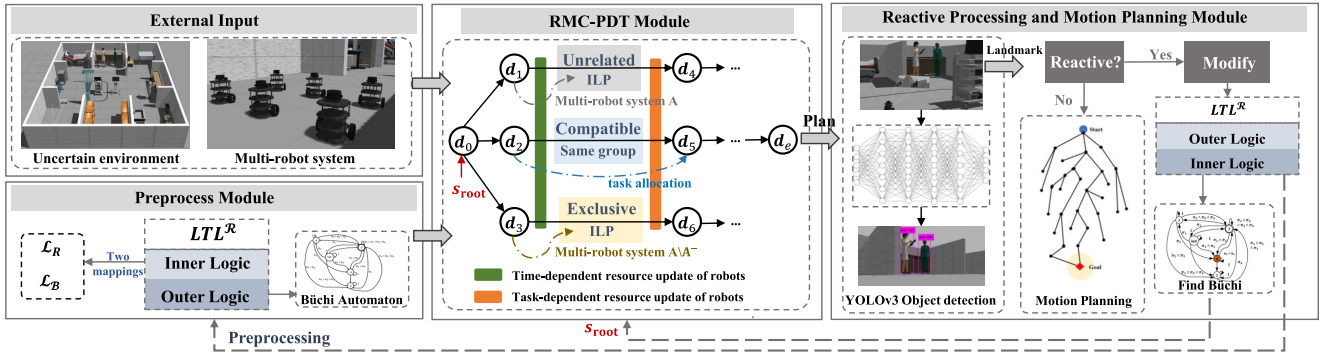


Fig. 1. Framework of reactive multiconstraint task allocation and planning, which mainly consists of four modules, namely preprocessing module, RMC-PDT module, reactive process module, and motion planning module.

The amount of resources required for the above tasks is listed in Table II. For instance, ϕ_1 requires the robots to carry at least 11 boxes of medicines, 12 L of disinfectant, and 102 mg of active reagent. Besides resources, there may exist inter-task constraints. Specifically, ϕ_1 and ϕ_2 are exclusive tasks since robots that have visited the ward are not allowed to visit the laboratory and disinfection room to avoid cross-infection. Differently, visiting the laboratory and the disinfection room are compatible tasks, and thus, after completing one task, the same group of robots can be reassigned for another compatible task. The above tasks with resource and inter-task constraints can be expressed by LTL^R formulas as $\phi_1 = \varphi_1^{R_1, +1}$, $\phi_2 = \varphi_2^{R_2, -1}$, $\phi_3 = \varphi_3^{R_3, +2}$ and $\phi_4 = \varphi_4^{R_4, +3}$, where $R_1 = \{(n_{LC}^C, 11), (n_{DC}^R, 12), (n_{VC}^R, 102)\}$, $R_2 \cdot v = (12, 10, 101)$, $R_3 \cdot v = (18, 18, 92)$ and $R_4 \cdot v = (14, 16, 100)$ indicate the resource requirements. The global task of the robots is then defined as $\phi_G = \bigwedge_{i=1}^4 \phi_i$. The local reactive tasks are activated based on the local event. For instance, during the execution of $\pi_{\text{central_laboratory}}$, if the landmark of “person” is detected, a temporary reactive task $\phi_L = \varphi_L^{R_L, +4}$ needs to be performed, where $\varphi_L = \square \diamond \pi_{\text{pharmacy_dispensary}} \wedge \square \diamond \pi_{\text{dressing_room}}$ and $R_L = \{(n_{LC}^C, 19), (n_{DC}^R, 13), (n_{VC}^R, 112)\}$. If the landmark “person” is detected in the doctor’s room, π_{X_ray} is no longer required and thus π_{X_ray} should be removed from the subsequent planning.

Assumption 1: There exists a plan Π satisfying the global task ϕ and potential temporary local task ϕ_L , and the resources possessed by robots are sufficient to complete tasks.

Due to the prefix-suffix structure of the accepting run from NBA, the plan can be represented as $\Pi = \Pi_{\text{pre}} \Pi_{\text{suf}} \Pi_{\text{suf}}, \dots$, which is composed of a finite prefix stage Π_{pre} and infinite suffix stages $\Pi_{\text{suf}} \Pi_{\text{suf}}, \dots$. If Π satisfies ϕ , a finite plan $\Pi^f = \Pi_{\text{pre}} \Pi_{\text{suf}}$, which is a fragment of Π , also satisfies ϕ under Assumption 1.

Problem 1: Given an uncertain semantic environment Env , a heterogeneous multirobot system A , and an LTL^R formula ϕ that encodes multiple constraints, our goal is to develop a reactive task allocation strategy that satisfies the task ϕ as well as the reactive local tasks ϕ_L in an uncertain semantic environment, i.e., finding a feasible finite plan $\Pi^f = \Pi_{\text{pre}} \Pi_{\text{suf}}$.

III. REACTIVE MULTICONSTRAINT TASK ALLOCATION

In this section, we present the developed reactive task allocations and planning framework that satisfies LTL^R formulas subject to multiple task constraints and time/task-dependent or

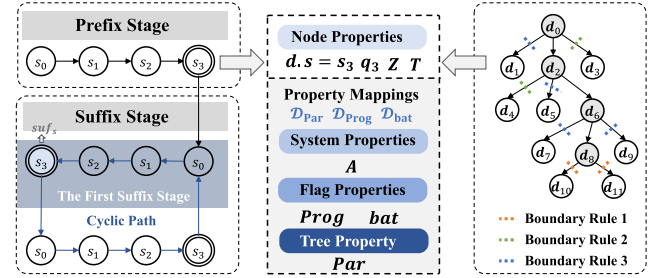


Fig. 2. Schematic of the planning decision tree.

inherent resources in an uncertain semantic environment. The framework contains four main modules as shown in Fig. 1, namely the preprocessing module, the RMC-PDT module, the reactive processing module, and the motion planning module.

Inspired by our previous works [26], we first introduce a planning decision tree to encode states of the system and tasks as illustrated in Fig. 2. To facilitate the analysis, a data structure is used throughout this work to represent the node properties, e.g., $d_i \cdot s$ means the automaton state of the node d_i .

Definition 3: The proposed planning decision tree $\mathcal{T}$ is constructed based on a set of nodes $\{d_i\}$, $i \in \mathbb{N} \cup \{0\}$, where d_0 represents the root of the tree. Each node in $\mathcal{T}$ is defined as a tuple $d_i = (s, q, Z, T)$, where $s \in S$ is the automaton state, q is the task state, Z is the task allocations corresponding to the task $\mathcal{L}_a(q)$ and $T = \{t_1, t_2, \dots, t_N\}$ is the estimated time of each robot completing $\mathcal{L}_a(q)$. Besides, there are four property mappings $\mathcal{D}_{\text{Par}}: \{d_i\} \rightarrow \text{Par}$, $\mathcal{D}_A: \{d_i\} \rightarrow A$, $\mathcal{D}_{\text{Prog}}: \{d_i\} \rightarrow \text{Prog}$, and $\mathcal{D}_{\text{bat}}: \{d_i\} \rightarrow \text{bat}$, where Par is the parent node, A is the multirobot system, bat is the task batch indicates the inter-task constraints, and $\text{Prog} = \{\text{pre}, \text{suf}_s, \text{suf}_{\text{cyc}}\}$ is the flag indicating the mission stage of node d_i , as shown in Fig. 2. Specifically, $\text{Prog} = \text{pre}$ means d_i is in the prefix stage, $\text{suf}_s \in S$ means the start and end states in the first suffix stage, and suf_{cyc} means the remaining cyclic path excluding the first suffix stage.

The proposed reactive multiconstraint task allocation method is built upon $\mathcal{T}$ in Definition 3, as detailed in Algorithm 1. Given an LTL^R formula, we first preprocess it to generate a conventional LTL formula φ_{ini} and two mappings for the extraction of resource and inter-task constraints (line 1), which is detailed in Section III-A. The generated LTL formula φ_{ini} is then translated to a Büchi automaton (line 2). The automaton and mappings generated from the preprocessing

Algorithm 1 Reactive Multiconstraint Task Allocation and Planning

Input: The initial LTL^R formula ϕ_{ini} , uncertain semantic environment Env , the multi-robot system A

Output: The final plan

```

1  $\varphi_{ini}, \mathcal{L}_R^{ini}, \mathcal{L}_B^{ini} \leftarrow \text{Preprocess}(\phi_{ini});$ 
2 Convert  $\varphi_{ini}$  to Büchi automaton  $B_{ini}$ ;
3  $\Pi_{ini}^* \leftarrow \text{RMC-PDT}(\mathcal{L}_R^{ini}, \mathcal{L}_B^{ini}, B_{ini}, Env, A, s_0);$ 
4 Execute according to  $\Pi_{ini}^*$ ;
5 if  $D_{landmark} > D_{threshold}$  and  $a_{detection} \cdot p$  in  $M_{task}$  then
6    $\varphi_{new}, \mathcal{L}_R^{new}, \mathcal{L}_B^{new} \leftarrow \text{Modify}(\varphi_{ini}, \mathcal{L}_R^{ini}, \mathcal{L}_B^{ini}, Q_{hasdone});$ 
7   Convert  $\varphi_{new}$  to Büchi automaton  $B_{new}$ ;
8    $S \leftarrow \text{Reachable\_States}(B_{new}, Q_{hasdone});$ 
9   for  $s_{new} \in S$  do
10     $\Pi_{new} \leftarrow \text{RMC-PDT}(\mathcal{L}_R^{new}, \mathcal{L}_B^{new}, B_{new}, Env, A, s_{new});$ 
11     $\Pi.append(\Pi_{new});$ 
12  $\Pi^* \leftarrow \arg \min_{\Pi} \text{Cost}(\Pi);$ 
13 Execute according to  $\Pi^*$ ;
```

module are leveraged in the RMC-PDT module for exploration and generation of an initial feasible plan Π_{ini}^* (line 3), which is fed into the motion planning module to obtain the executable path of the robotic system and the robots then execute according to the plan Π_i^* (line 4). Once a robot with perception capability, denoted as $a_{detection}$, detects a landmark indicating a temporary local task with a probability greater than a prespecified threshold $D_{threshold}$ according to YOLO network, while this robot is currently in this specified task region $M_{task} \subset M$, the reactive processing module (detailed in Section III-C) will be activated (lines 5–11). At the same time, the robotic system will stop executing the next task, while the states of multirobot system, environment, and adding/subtracting task items are updated to the RMC-PDT module. The modified LTL^R formula ϕ_{new} and the sets of reachable Büchi states obtained from the reactive process module are then used in RMC-PDT to explore and obtain a set of new reachable plans Π (lines 6–11). Finally, the plan $\Pi^* \in \Pi$ with the lowest cost (i.e., the task completion time) is selected as the subsequent behaviors to be implemented (lines 12 and 13).

Remark 2: To avoid interference caused by noise or false detections, the detection part needs to meet the following conditions to trigger the subsequent reactive planning module: 1) the detection confidence of the landmark exceeds our predefined threshold $D_{threshold}$; 2) the landmark is located within the robot's current task region; and 3) the detected category matches the predefined semantics associated with the corresponding dynamic tasks.

A. Preprocessing Module

Since $\varphi^{R,B}$ comprises the task and its corresponding resource and inter-task constraints, the goal of preprocessing module is to extract them to facilitate the subsequent planning. To extract the task, we simply extract φ from $\varphi^{R,B}$. For the constraints, we develop two mappings, $\mathcal{L}_R(\cdot)$ and $\mathcal{L}_B(\cdot)$, which map an atomic proposition to its resource requirements and inter-task constraints, respectively. For example, consider an LTL^R specification $\phi_G = \phi_1 \wedge \phi_2$, where $\phi_1 = \square\Diamond(\pi_1 \wedge \pi_2)^{R_1, b_1}$ and $\phi_2 = (\square\Diamond\pi_3 \wedge \square\Diamond\pi_4)^{R_2, b_2}$. The extracted subtasks are $\square\Diamond(\pi_1 \wedge \pi_2)$ and $(\square\Diamond\pi_3 \wedge \square\Diamond\pi_4)$, which yield the conventional LTL formula $\varphi = \square\Diamond(\pi_1 \wedge \pi_2) \wedge (\square\Diamond\pi_3 \wedge \square\Diamond\pi_4)$. The extracted

task constraints are $\mathcal{L}_R(\pi_1) = \mathcal{L}_R(\pi_2) = R_1$, $\mathcal{L}_R(\pi_3) = \mathcal{L}_R(\pi_4) = R_2$, $\mathcal{L}_B(\pi_1) = \mathcal{L}_B(\pi_2) = b_1$, and $\mathcal{L}_B(\pi_3) = \mathcal{L}_B(\pi_4) = b_2$.

Lemma 1: The preprocessing process does not change the resource and inter-task requirements of LTL^R.

Proof: We first show that the preprocessing module does not affect the task operation of LTL^R. According to the preprocessing mechanism, the LTL formula is extracted from LTL^R by simply removing the resource and inter-task constraints. Hence, the extracted LTL task has exactly the same temporal and logic order of LTL^R. Additionally, since the mappings of resources and batches do not affect each other ($\mathcal{L}_R$ and $\mathcal{L}_B$ correspond to two different types of task constraints), the extracted resource and inter-task constraints are consistent with those of LTL^R. ■

B. RMC-PDT Module

The RMC-PDT module is designed to generate a feasible plan Π . Since the planning procedure is based on the proposed planning decision tree $\mathcal{T}$, we first present the construction of $\mathcal{T}$. Given the mappings $\mathcal{L}_R$ and $\mathcal{L}_B$, and the automaton state of the root node d_0 , we first initialize the root node as $d_0 \cdot q = q_0$, $d_0 \cdot Z = [0, 0, \dots, 0]$, $d_0 \cdot T = \{0, 0, \dots, 0\}$, $\mathcal{D}_{Par}(d_0) = \emptyset$, $\mathcal{D}_{Prog}(d_0) = pre$, and $\mathcal{D}_{bat}(d_0) = \mathcal{L}_B(\mathcal{L}_a(q_0))$ (line 1). Then, starting from the root node d_0 , the tree $\mathcal{T}$ is expanded by traversing and adding nodes as detailed in Algorithm 3. Let $\mathcal{T}_{tra}$ be the set of nodes that have already been traversed. The tree will explore nodes that have not been traversed. For each exploring node, the set of its expandable subnodes is denoted as the subtree $\mathcal{T}_{sub}$, which is initialized as an empty set (line 4). Let $\Gamma_i = d_0, \dots, d_{i-1}$ be the path from d_0 to d_i excluding d_i in $\mathcal{T}$ and $\{\Gamma_i \cdot s\}$ be the set of automaton states of the nodes in Γ_i . For any $s \in S \setminus \{\Gamma_i \cdot s\}$, RMC-PDT algorithm finds all task states q satisfying $\mathcal{L}_a(q) \in \delta(d_i \cdot s, s)$ to create the subnode d_i^{sub} (lines 5–7). The mission stage of d_i^{sub} is obtained by

$$f_{Prog}(Prog, s) = \begin{cases} Prog, & \text{if } s \notin \mathcal{F} \\ suf_s, & \text{if } s \in \mathcal{F}, \quad Prog = pre \\ suf_{cyc}, & \text{if } Prog = s. \end{cases} \quad (2)$$

The generated subnode d_i^{sub} is added to the subtree (lines 7–13), and $\{\Gamma_i \cdot s\}$ is updated according to the mission stage of the subnode (lines 9–12). If $\mathcal{D}_{Prog}(d_i^{sub}) = \mathcal{D}_{Prog}(d_i)$, it means that the current mission stage has not been completed yet and the current automaton state s should be added to $\{\Gamma_i \cdot s\}$. Otherwise, the current mission stage is completed. Thus $\{\Gamma_i \cdot s\}$ is set as an empty set for starting a new mission stage of the exploration process. After traversing all the subnodes corresponding to q that satisfy $\mathcal{L}_a(q) \in \delta(d_i \cdot s, s)$, $s \in S \setminus \{\Gamma_i \cdot s\}$, RMC-PDT prunes subtrees using boundary rules as follows, which are developed to prune the nodes and branches that do not contribute to the task allocation.

- 1) For any node d in $\mathcal{T}$, if $\mathcal{D}_{Prog}(d) = suf_{cyc}$, d will no longer be traversed.
- 2) RMC-PDT removes nodes whose mission stage remains invariant, and the automaton state has previously been traversed.
- 3) For any two nodes d_1 and d_2 in $\mathcal{T}$, if $d_1 \cdot s = d_2 \cdot s$ and $\mathcal{D}_{Prog}(d_1) = \mathcal{D}_{Prog}(d_2)$, the node with larger cost will no longer be traversed.

Then the pruned subtree is added to $\mathcal{T}$ and d_i is added to the traversed set (lines 14 and 15). Finally, the branch

Algorithm 2 RMC-PDT

Input: The mappings $\mathcal{L}_R$ and $\mathcal{L}_B$, uncertain semantic environment Env , the multi-robot system A , and the automaton state of the root node

Output: Plan Π

- 1 Initialize $\mathcal{T} \leftarrow \{d_0\}$, where $d_0.s = s_{root}$;
- 2 **while** true **do**
- 3 **for** $d_i \in \mathcal{T}$ **and** $d_i \notin \mathcal{T}_{tra}$ **do**
- 4 Initialize sub-tree $\mathcal{T}_{sub}$;
- 5 **for** $s \in S \setminus \{\Gamma_i.s\}$ **do**
- 6 **for** q such that $\mathcal{L}_a(q) \in \delta(d_i.s, s)$ **do**
- 7 $d_i^{sub} \leftarrow$
- 8 Tree_Expansion($q, d_i, A, \mathcal{L}_R, \mathcal{L}_B, \mathcal{T}_{sub}$);
- 9 $\mathcal{D}_{prog}(d_i^{sub}) \leftarrow f_{prog}(\mathcal{D}_{prog}(d_i), s)$;
- 10 **if** $\mathcal{D}_{prog}(d_i^{sub}) = \mathcal{D}_{prog}(d_i)$ **then**
- 11 $\{\Gamma_i.s\}.append(s)$;
- 12 **else**
- 13 Set $\{\Gamma_i.s\}$ to the empty set;
- 14 $\Gamma_i.append(d_i^{sub})$ and $\mathcal{T}_{sub}.append(d_i^{sub})$;
- 15 Traverse $d_i^{sub} \in \mathcal{T}_{sub}$ and prune $\mathcal{T}_{sub}$ according to the Boundary Rules;
- 16 $\mathcal{T}.append(\mathcal{T}_{sub})$ and $\mathcal{T}_{tra}.append(d_i)$;
- 17 Select $d^* = \text{argmin}_{d_i \in \mathcal{T}_B} J$ and $d^*.Prog = suf_{cyc}$;
- 18 Return $\Pi = (\mathcal{T}^*.s, \mathcal{T}^*.q, \mathcal{T}^*.Z)$;

Algorithm 3 Tree_Expansion

Input: The task state q , the node d_i , the multi-robot system A , the capability mapping $\mathcal{L}_R$, the task batch mapping $\mathcal{L}_B$, and the sub-tree $\mathcal{T}_{sub}$.

Output: The sub-node d_i^{sub} .

- 1 Initialize sub-node d_i^{sub} ;
- 2 Update time-dependent resources possessed by robots in A ;
- 3 Set $A_p \leftarrow A$;
- 4 **if** $d_i^{sub}.q$ has a compatible constraint **then**
- 5 $d_i^{sub}.Z$ is assigned to the task allocation of the node with the same task batch in $\mathcal{T}_{sub}$;
- 6 **else**
- 7 **if** $d_i^{sub}.q$ has an exclusive constraint **then**
- 8 Set $A_p \leftarrow A \setminus A^-$;
- 9 $d_i^{sub}.Z \leftarrow \text{ILP}(\mathcal{L}_R(\mathcal{L}_a(q)), A_p)$ by (3);
- 10 **for** $a_n \in A$ **which satisfies** $d_i^{sub}.Z(n) = 1$ **do**
- 11 Update task-dependent resources of robot a_n ;
- 12 Set $t_n^i = \max_{m: d_i^{sub}.Z(m)=1} t_m^i$ and $p_{a_n}^i = \mathcal{L}_m(q)$;
- 13 $d_i^{sub}.t_n^i = \max\{t_n^i, t_n^{i-1}\}$;
- 14 Return d_i^{sub} ;

corresponding to the node d^* with the minimum cost and mission stage suf_{cyc} is selected (line 16). Starting from d^* and tracing back until the root node, we obtain the resulting tree $\mathcal{T}^*$. The sequences of automaton states, task states, and task allocations can be extracted from $\mathcal{T}^*$ as $\mathcal{T}^*.s$, $\mathcal{T}^*.q$, and $\mathcal{T}^*.Z$, respectively, which collectively form the feasible plan Π^* , where the cost is the maximum task completion time, i.e., $J = \max\{d_i \cdot T\}$ (line 17).

Lemma 2: The boundary rules do not affect the feasibility of the planning of RMC-PDT.

Proof: Since the first boundary rule aims to stop traversal to generate $\Pi^f = \Pi_{pre}\Pi_{suf}$, it does not affect the feasibility in Π^f . The second and third boundary rules prune redundant

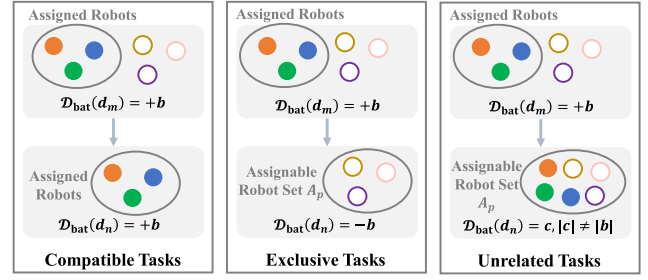


Fig. 3. Illustration of decision-making under different inter-task constraints corresponding to different task batch values.

nodes that do not contribute to the planning, i.e., the nodes with invariant mission stage and traversed automaton state, or the nodes with larger costs among the nodes of the same automaton state in the same mission stage. Thus, the feasibility of the generated plan is not affected by the boundary rules. ■

Tree expansion in Algorithm 3 is a core component in the RMC-PDT module. Specifically, given a task state q , we first initialize the subnode d_i^{sub} as $d_i^{sub}.s = d_i.s$, $d_i^{sub}.q = q$ and $d_i^{sub}.Z = [0, 0, \dots, 0]$, while its corresponding task batch is $\mathcal{L}_B(\mathcal{L}_a(q))$ and its parent node is d_i . Thus, we set $d_i^{sub}.T = d_i.T$ and $\mathcal{D}_{prog}(d_i^{sub})$ is initialized the same as the parent node d_i . After updating the values of time-dependent resources possessed by robots (line 2), we define a variable A_p initialized as the multirobot system A for subsequent task allocation (line 3). The inter-task constraints corresponding to nodes are determined (lines 4–9). If there exists a node d_s with the same task batch in $\mathcal{T}_{sub}$ (i.e., $\mathcal{D}_{bat}(d_i^{sub}) = \mathcal{D}_{bat}(d_s)$) and the task allocation of d_s can satisfy the task requirement of $d_i.q$ (i.e., the resources possessed by the assigned robots are sufficient to meet the requirements of the task $d_i.q$), the tasks corresponding to d_i and d_s are mutually compatible. This implies that the tasks corresponding to d_i^{sub} and d_s can be assigned to the same group of robots. Thus the task allocation of d_s can be directly assigned to d_i^{sub} , i.e., $d_i^{sub}.Z = d_s.Z$ (lines 4 and 5). If there exists a node d_s with the task batch opposite to d_i^{sub} in $\mathcal{T}_{sub}$ (i.e., $\mathcal{D}_{bat}(d_i^{sub}) = -\mathcal{D}_{bat}(d_s)$), the tasks of d_i^{sub} and d_s are mutually exclusive, which means that d_i^{sub} cannot employ the robots assigned by d_s . Therefore, we set $A_p = A \setminus A^-$, where A^- is the robots that have been assigned in d_s (line 8). If d_i does not satisfy the above conditions, the task of d_i^{sub} does not have any inter-task constraints and is an unrelated task. In this case, $A_p = A$. Fig. 3 shows how different task batch values lead to different decision-making branches. The following ILP formulation is solved for the task allocation Z of d_i (line 9):

$$\begin{aligned}
 & \min_Z \sum_{n=1}^N d_i^{sub}.Z(n) \times t_n^i \\
 & s.t. \sum_{n=1}^N d_i^{sub}.Z(n) \times a_n \cdot \tilde{R}_k \geq \mathcal{L}_{R_k}(\mathcal{L}_a(q)) \\
 & k \in [N_C], \quad a_n \in A_p
 \end{aligned} \tag{3}$$

where the estimated time for robot a_n at node d_i^{sub} is given by $t_n^i = t_n^{i-1} + (1/v_a) \|p_{a_n}^{i-1} - \mathcal{L}_m(q)\|_2$, i.e., the sum of the estimated time at the previous node and straight-line travel time from the current position of a_n to the task location. It can be seen that the estimated time serves as a lower bound for determining task allocation during the RMC-PDT, which is used as an input to the ILP formulation, rather than representing the robot's

actual operation time. If a robot is assigned to perform the task corresponding to $d_i^{\text{sub}} \cdot q$ (i.e., a robot a_n with $d_i^{\text{sub}} \cdot Z(n) = 1$) and the resource is task-dependent, the robot's resource has to be updated along the task execution (line 11). The time stamp is set as the maximum task completion time of all robots assigned to perform $d_i^{\text{sub}} \cdot q$ (i.e., $\max_{m: d_i^{\text{sub}} \cdot Z(m)=1} t_m^i$), so that robots will wait until all assigned robots reach $\mathcal{L}_m(d_i^{\text{sub}} \cdot q)$. Besides, the estimated positions of assigned robots are updated to $\mathcal{L}_m(d_i^{\text{sub}} \cdot q)$ (lines 10–12). Since time stamps of nonassigned robots remain the same as the estimated time of the previous node, the time stamp of d_i^{sub} is the maximum of the estimated time of all robots, i.e., $d_i^{\text{sub}} \cdot t_n^i = \max\{t_n^i, t_n^{i-1}\}$ (line 13).

Theorem 1: RMC-PDT is valid, i.e., the task allocation generated by RMC-PDT satisfies the LTL^R formula.

Proof: According to the mechanism of RMC-PDT, the Büchi automaton is encoded in the proposed planning decision tree, which is expanded through reachable automaton states to ensure that the plan generated by RMC-PDT satisfies the task requirements. Thereafter, RMC-PDT focuses on three cases (i.e., unrelated, compatible, and exclusive cases) for exploration according to inter-task constraints. For unrelated cases, RMC-PDT directly solves the problem by ILP and the multirobot system A . Since the constraints of ILP ensure that the resources available to the assigned robots are not less than those required by tasks, the solution of the unrelated case is valid. For compatible cases, RMC-PDT first evaluates the robots assigned for the corresponding compatible task. If the resources possessed by these robots fulfill the task requirements, the compatible case is performed, and these robots are directly assigned to this task. If not, the unrelated case is explored to search for a new task allocation that satisfies the resource requirements of the task. For exclusive cases, the robots assigned to mutually exclusive tasks are first removed from the multirobot system A . The updated multirobot system A_p is then used in the ILP to ensure that the robots already assigned to corresponding exclusive tasks will not be selected, while ILP ensures that the resource requirements are fulfilled. In summary, the inter-task constraint and resource requirements are guaranteed to be satisfied in the above three cases. Also, the Büchi automaton corresponding to the preprocessed traditional LTL formula guarantees the temporal requirements for tasks. Thus, RMC-PDT is valid. ■

Theorem 2: The RMC-PDT is complete, i.e., if a feasible solution exists, RMC-PDT is guaranteed to find it.

Proof: To prove that RMC-PDT is complete, we need to show the completeness of task allocation and mission planning, respectively. By the Assumption 1, there exist sufficient resources possessed by robots for task allocation. Under this assumption, the ILP in RMC-PDT can definitely find a feasible solution. Therefore, RMC-PDT is complete for task allocation. For the mission planning, Lemma 2 first proves that the boundary rules have no effect on the completeness of mission planning. Suppose that there exists a plan $\Pi^{\text{fea}} = \Pi_{\text{pre}}^{\text{fea}} \Pi_{\text{suf}}^{\text{fea}}, \dots$ satisfying the LTL^R formula with sequences of automaton states, task states and task allocations as $s^{\text{fea}} = s_0, \dots, s_j$, $q^{\text{fea}} = q_0, \dots, q_j$ and $Z^{\text{fea}} = Z_0, \dots, Z_j$, respectively, where $\mathcal{L}_a(q_{i+1}) = \pi_{i+1}$, $\pi_{i+1} = \delta(s_i, s_{i+1})$ and $i \in [n-1]$. According to the mechanism of RMC-PDT, starting the exploration from the root node of $\mathcal{T}$, it will traverse all task states q that satisfy $\mathcal{L}_a(q) \in \delta(d_i \cdot s, s)$, where $s \in S \setminus \{\Gamma_i \cdot s\}$, thus there will always exist a traversed node with $d_i \cdot s = s_k \in s^{\text{fea}}$, $d_i \cdot q = q_k \in q^{\text{fea}}$ and $d_i \cdot Z = Z_k \in Z^{\text{fea}}$. Similarly, the node

Algorithm 4 Reachable States

Input: Büchi automaton B_{new} and current completed tasks

Q_{hasdone}

Output: The reachable state set $\mathcal{S}$

```

1 Initialize  $\text{büchi} \leftarrow [s_0]$ ,  $\text{result} \leftarrow \emptyset$ ;
2 for  $\mathcal{L}_a(q) \in Q_{\text{hasdone}}$  do
3   for  $s \in \text{büchi}$  do
4      $\perp$  add  $s'$  into  $\text{result}$  for  $\forall s', q \in B_{\text{new}} \cdot \delta(s, s')$ ;
5    $\text{büchi} \leftarrow \text{result}$ ;
6    $\text{result} \leftarrow \emptyset$ ;
7  $\mathcal{S} \leftarrow \text{büchi}$ ;
8 Return  $\mathcal{S}$ ;
```

d_{end} with $d_{\text{end}} \cdot s = s_n$, $d_{\text{end}} \cdot q = q_n$ and $d_{\text{end}} \cdot Z$ will be traversed during the exploration of $\mathcal{T}$. Therefore, every node corresponding to the state of Π^{fea} will be traversed, i.e., Π^{fea} will definitely be found if it exists without using boundary rules. Finally, the RMC-PDT prunes the nodes through the boundary rules, which do not affect the feasibility of the plan according to Lemma 2. Therefore, RMC-PDT is complete. ■

C. Reactive Process Module

In this section, an automaton-reconstruction-based method is presented as a key component of the reactive process module, enabling timely and adaptive task planning to accommodate evolving task requirements. The reactive process module receives inputs comprising the conventional LTL formula, the extracted resources, inter-task constraints from the preprocessing module, and any newly added or removed local tasks. A revised LTL formula is reconstructed into a new automaton, and updated constraints are then output to facilitate reactive planning. In particular, to enable reactive planning, the queue of completed tasks Q_{hasdone} is first obtained, where the i th element of Q_{hasdone} is denoted as $Q_{\text{hasdone}}[i]$. The original LTL formula φ_{ini} is then updated based on Q_{hasdone} and the local task ϕ_L . Specifically, the addition of a new local task is to insert ϕ_L to φ_{ini} according to the task progress Q_{hasdone} , while the removal of a task is simply to delete the corresponding atomic propositions. Meanwhile, we obtain two modified mappings $\mathcal{L}_R^{\text{new}}$ and $\mathcal{L}_B^{\text{new}}$ based on the corresponding added or removed local tasks. For instance, reactive planning can be activated based on the detection of desired landmarks. In Example 2, if a “person” is detected in the execution of $\pi_{\text{central_laboratory}}$, the predefined local task $\phi_L = \varphi_L^{R,B}$ with $\varphi_L = \Box \Diamond \pi_{\text{pharmacy_dispensary}} \wedge \Box \Diamond \pi_{\text{dressing_room}}$ is activated immediately. The executed formula is modified to $\varphi_{\text{new}} = \Box \Diamond \pi_{\text{ward}} \wedge \Box \Diamond \pi_{\text{central_laboratory}} \wedge \varphi_L \wedge \Box \Diamond \pi_{\text{disinfection_room}} \wedge \Box \Diamond \pi_{\text{waiting_room}} \wedge \Box \Diamond \pi_{\text{X_ray}} \wedge \Box \Diamond \pi_{\text{doctor's_room}}$. The relevant mappings about ϕ_L are added to $\mathcal{L}_R^{\text{new}}$ and $\mathcal{L}_B^{\text{new}}$ as well.

Let B_{new} denote the NBA corresponding to φ_{new} , which is the reconstructed automaton. Based on the updated formula φ_{new} , the next is to determine a set of reachable automaton states, denoted as $\mathcal{S}$, that the completed task states can reach in B_{new} . As shown in Algorithm 4, we first initialize a queue of automaton states $\text{büchi} = [s_0]$ and an intermediate queue $\text{result} = \emptyset$ (line 1). Then, we traverse the tasks in Q_{hasdone} in turn, thus obtaining the reachable automaton states (lines 2–6). We sequentially find the reachable states of Q_{hasdone} . For each state in büchi , we find the state s' that can be reached from the current Büchi state s through $q \in Q_{\text{hasdone}}$, i.e., s' is a possible Büchi state that can be reached by completing

q . Hence, s' is added to the queue *result* (lines 3–5). Then, we set *result* to be the empty set (line 6). Repeat the above steps, and finally, we can find the set of reachable states $\mathcal{S}$ that B_{new} may reach after completing Q_{hasdone} (line 7). In the following process, $\mathcal{S}$ will be used in the new round of RMC-PDT exploration by using all possible reachable Büchi states as the automaton states of the root node in the tree, and thus expanding this tree.

Lemma 3: The automaton states obtained by Algorithm 4 are the feasible states that the current robotic system can reach from its initial state.

Proof: First, Algorithm 4 finds possible reachable automaton states after completing $Q_{\text{hasdone}}[0]$ from the initial automaton state according to the transition function $B_{\text{new}} \cdot \delta$, thus the possible reachable automaton states should satisfy $\mathcal{L}_a(Q_{\text{hasdone}}[0]) \in B \cdot \delta(s_0, s_{\text{reachable}})$. Then the possible reachable states are added to *büchi* and considered as the current automaton states to search for the next possible reachable automaton states after executing $Q_{\text{hasdone}}[1]$. By repeating the process above, since the search follows the transition function in B_{new} , which guarantees the reachability of the obtained automaton states, the set of automaton states obtained by Algorithm 4 is the collection of all possible states that can be reached by robot system after completing Q_{hasdone} from the initial state. ■

D. Discussion

Lemma 4: The plan Π obtained by Algorithm 1 satisfies $\text{LTL}^{\mathcal{R}}$ and the reactive task requirements.

Proof: According to Theorems 1 and 2, it can be concluded that if there exists a feasible solution, RMC-PDT is guaranteed to find the solution, which satisfies the mission planning and task requirements of $\text{LTL}^{\mathcal{R}}$. Meanwhile, the planning decision tree $\mathcal{T}$ guarantees that the task allocations, automaton states, and task states in the node are one-to-one, thus the resulting plan satisfies the requirements of $\text{LTL}^{\mathcal{R}}$. For the reactive planning, according to Lemma 3, it can be obtained that Algorithm 4 can find all automaton states reachable from the current state. Once a task-related landmark is detected, the corresponding reactive plan will invoke RMC-PDT (line 10 in Algorithm 1). Owing to Theorems 1 and 2, the resulting solution will satisfy the new task requirements. This concludes the proof. ■

To show the complexity of reactive multiconstraint task allocation, we first consider the complexity of the RMC-PDT module. According to the boundary rules, the expansion of $\mathcal{T}$ stops when the mission stage of node is $\text{suffix}_{\text{cyc}}$, which means that this tree has traversed the nodes of the prefix stage and the first cyclic path. Thus, the length of $\mathcal{T}$ is at most $2|\mathcal{S}|$. The boundary rule (3) ensures that $\mathcal{T}$ has a maximum width of $(1 + |\mathcal{F}|) \times |\mathcal{S}|$ and each node can generate at most $|\mathcal{S}| \times |\mathcal{Q}|$ subnodes. Therefore, the whole tree generates no more than $2|\mathcal{S}| \times (1 + |\mathcal{F}|) \times |\mathcal{S}| \times |\mathcal{S}| \times |\mathcal{Q}|$ nodes. It is assumed that the tasks corresponding to each node are either unrelated or mutually exclusive (i.e., the most complex case), and each node needs to solve an ILP problem, which corresponds to a complexity of $\mathcal{O}(N_c \times 2^N)$. Since the number of $\mathcal{F}$ is much smaller than $\mathcal{S}$, the overall complexity of RMC-PDT is $\mathcal{O}(2^N \times |\mathcal{S}|^2 \times |\mathcal{S}| \times |\mathcal{Q}|)$. In general, the number of generated subnodes is linearly related to $|\mathcal{Q}|$, thus the above complexity can be further simplified to $\mathcal{O}(2^N \times |\mathcal{S}|^2)$. Next, we prove the complexity of our framework of reactive multiconstraint task allocation. Assuming that N_r times reactive

TABLE III
COMPUTATIONAL TIME FOR DIFFERENT NUMBERS OF ROBOTS
WITH DIFFERENT TYPES OF RESOURCES

Number of Robots	Time(s) (inherent)	Time(s) (time – dependent)	Time(s) (task – dependent)	Time(s) (combined)
6 (2)	0.1350	0.1437	0.1379	0.1421
15 (5)	0.1427	0.1406	0.1397	0.1487
60 (20)	0.1744	0.1758	0.192	0.2011
150 (50)	0.2339	0.2403	0.2359	0.2388
300 (100)	0.3303	0.3468	0.3482	0.3378
3000 (1000)	2.5021	0.7929	2.7332	2.7211
30000 (10000)	63.4257	64.1890	62.0334	64.4868

planning is performed, i.e., N_r landmarks trigger the module of reactive planning, the entire framework performs RMC-PDT in $N_r + 1$ times. Therefore, the complexity of our method is $\mathcal{O}((N_r + 1) \times 2^N \times |\mathcal{S}|^2)$.

In general, the reactive planning module is activated immediately upon landmark detection, prompting the robotic system to execute the newly generated plan without delay. However, in certain scenarios, immediate reactive planning may not be necessary. Instead, it may be required at a specific time following the detection of a particular landmark. For instance, when a landmark is identified that necessitates the addition or removal of tasks after completing a pending task, reactive planning should be delayed. To address this, our method modifies Q_{hasdone} , where Q_{hasdone} represents not the set of completed tasks, but the sequence of tasks from the initial task to the specific task in question.

Remark 3: As defined in Section II-A, the resources available to robots are limited within the range $[0, \text{Cap} \cdot v]$. However, during task execution, there are instances where a particular resource is nearly depleted or almost reaching its maximum capacity. To address these situations, we propose two strategies. The first strategy involves designating a location as a resource supply station when a robot's resource is about to deplete, which is equivalent to reactively adding a task to replenish the resource. Conversely, when a robot's resource is close to exceeding its capacity, subsequent tasks are configured to prevent further increases in that resource. The second strategy treats the robot as faulty when its resource is nearly exhausted, thereby excluding it from subsequent task assignments. Similarly, if a robot's resource is close to exceeding its capacity, it will not be assigned tasks that would further increase the resource, i.e., this robot will be placed in A^- of the corresponding task.

IV. EXPERIMENT

In our experiment, MATLAB R2019b is utilized for numerical experiments, and Ubuntu 18.04 and ROS Melodic are utilized for experiments. We evaluate the following features.

- 1) *Efficient Reactive Planning:* Assessing whether our method can efficiently address task planning and promptly respond to local temporary task requests.
- 2) *Robust Handling of Multiconstraint Tasks:* Evaluating whether our method can handle complex multiconstraint tasks for heterogeneous multirobot systems.
- 3) *Scalability:* Determining whether our method can be extended to large-scale robot systems, various resources, and inter-task constraints.

A. Numerical Experiment

1) *Different Numbers of Robots:* Consider an environment containing 8 task regions, corresponding to atomic

TABLE IV
COMPUTATIONAL TIME FOR DIFFERENT NUMBERS OF ROBOTS
PERFORMING DIFFERENT TYPES OF INTER-TASK CONSTRAINTS

Number of Robots	Time(s) (unrelated)	Time(s) (compatible)	Time(s) (exclusive)	Time(s) (combined)
6 (2)	0.1350	0.1251	0.2091	0.1964
15 (5)	0.1427	0.1339	0.2211	0.2061
60 (20)	0.1744	0.1688	0.2522	0.2421
150 (50)	0.2339	0.2291	0.3228	0.2957
300 (100)	0.3303	0.3151	0.4343	0.4057
3000 (1000)	2.5021	2.4980	2.5872	2.5348
30000 (10000)	63.4257	62.5649	66.3730	74.4459

propositions $\{\pi_i\}_{i \in [8]}$, where the multirobot system comprises three distinct categories $\{A_j\}_{j \in [3]}$. Robots in each category are equipped with three different capabilities, i.e., $A_1 \cdot \bar{R} = \{(n_1^X, 6), (n_2^X, 70), (n_3^X, 6)\}$, $A_2 \cdot \bar{R} = \{(n_1^X, 5), (n_2^X, 50), (n_3^X, 7)\}$ and $A_3 \cdot \bar{R} = \{(n_1^X, 7), (n_2^X, 50), (n_3^X, 5)\}$, where X can be either $\mathcal{R}$ or $\mathcal{C}$ depending on the task settings. The robots are assigned the global task $\phi = (\square\Diamond\pi_1)^{R_1, b_1} \wedge (\square\Diamond\pi_2)^{R_2, b_2} \wedge (\square\Diamond\pi_3)^{R_3, b_3}$. Additionally, we consider a temporary local task $\phi_L = (\square\Diamond\pi_4 \wedge \square\Diamond\pi_5)^{R_L, b_L}$, which should be executed after the completion of π_2 . The resources required for π_1 , π_2 , π_3 , and ϕ_L are (14, 113, 16), (9, 100, 12), (14, 113, 16), and (15, 123, 15), respectively. In the experiments, the number of robots varies from 6 to 30 000, with three categories equally divided. Each case is tested 20 times, and average computation times under different types of resources and different inter-task constraints are shown in Tables III and IV, respectively. In Table III, the tasks are set as unrelated, and the resources possessed by robots are set as inherent, time-dependent, task-dependent, or combined, respectively. In Table IV, the resources possessed by robots are set as inherent, which means all resources do not change over time or tasks. In addition, the inter-task constraints are set as unrelated, compatible, exclusive, and combined by changing $\mathcal{B}$ (i.e., $\{b_i\}_{i \in \{1,2,3,L\}}$) of tasks.

The results demonstrate that our method can efficiently generate a feasible plan for a large-scale multirobot system (i.e., in 1 min for 30 000 robots). When considering a relatively small-scale multirobot system (from 6 and 300 robots), our method can generate a feasible plan in less than a second. Besides, the computation time of our method is not affected by the change of resource constraints, owing to the mechanism of Algorithm 1 for updating the values of robots' possessed resources. When there are exclusive tasks, the computation time is slightly longer due to the increased task complexity. Nevertheless, when there is a compatible task property, the computation time is slightly shorter since the compatible tasks can employ the same group of robots without computation.

2) *Addition of Local Tasks*: To show the reactive performance of our method with the addition of local tasks, we continue with the experiment and system setup in Section IV-A1. Assume there are 15 robots equally divided into 3 categories, where the resource possessed by robots can be represented as $a_{1:5} \cdot \bar{R} = A_1 \cdot \bar{R}$, $a_{6:10} \cdot \bar{R} = A_2 \cdot \bar{R}$ and $a_{11:15} \cdot \bar{R} = A_3 \cdot \bar{R}$. The robots are required to complete the following task $(\Diamond\pi_1 \wedge \Diamond\pi_2)^{R_1, b_1} \wedge \Diamond\pi_3^{R_2, b_2}$, where the required amount of resources is $R_1 \cdot v = (28, 226, 32)$ and $R_2 \cdot v = (18, 200, 24)$. In this section, we consider the addition of temporary local tasks with different number of automaton states, which are $\phi_1^L = (\Diamond\pi_4)^{R_1, b_1}$, $\phi_2^L = (\Diamond\pi_4 \wedge \Diamond\pi_5)^{R_2, b_2}$, $\phi_3^L = (\Diamond\pi_4 \wedge \Diamond\pi_5 \wedge \Diamond\pi_6)^{R_3, b_3}$, $\phi_4^L = (\Diamond\pi_4 \wedge \Diamond\pi_5 \wedge \Diamond\pi_6 \wedge \Diamond\pi_7)^{R_4, b_4}$ and $\phi_5^L = (\Diamond\pi_4 \wedge \Diamond\pi_5 \wedge \Diamond\pi_6 \wedge \Diamond\pi_7 \wedge \Diamond\pi_8)^{R_5, b_5}$, while their

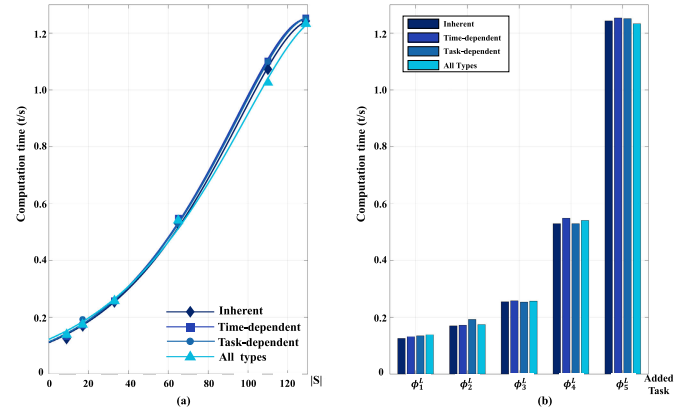


Fig. 4. (a) Computation time under different resource requirements with respect to the number of automaton states and (b) set of local tasks.

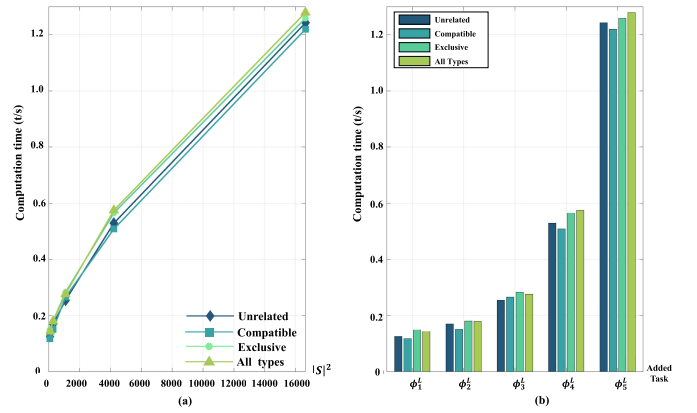


Fig. 5. (a) Computation time under different inter-task constraints with respect to the number of automaton states and (b) set of local tasks.

corresponding number of automaton states are 2, 4, 8, 16, and 32, respectively. These local tasks should be executed after the completion of π_2 , and the required resources for newly added tasks are all set as (22, 184, 22). We then evaluate the computation time in different kinds of scenarios, i.e., inherent, time-dependent, task-dependent, and combined resources under unrelated tasks (see Fig. 4) as well as unrelated tasks, compatible tasks, exclusive tasks, and combined, where robots' resources are inherent (see Fig. 5). It is evident that as the number of automaton states of the local task increases, the computation time increases correspondingly. The computation time exhibits a quadratic increase with respect to the number of automaton states. However, the type of resources used does not impact the overall computation time. On the other hand, the addition of exclusive tasks does increase the computation time, as analyzed in Section IV-A1. Nevertheless, our method still accommodates sudden requests within a short time.

3) *Removal of Tasks*: We consider a base formula $\phi_G = \bigwedge_{i=1}^8 \phi_i$ with $\phi_i = (\square\Diamond\pi_i)^{R_i, b_i}$, and evaluate the computation time under removal of different subtasks. The setups of the robot system and scenario are consistent with Section IV-A2. During the execution of the task, a signal is sent to the robot indicating that a certain task needs to be removed in the subsequent task, and the computational time under different resources and inter-task constraints are presented in Tables V and VI, derived from averaging the results of 20 experiments.

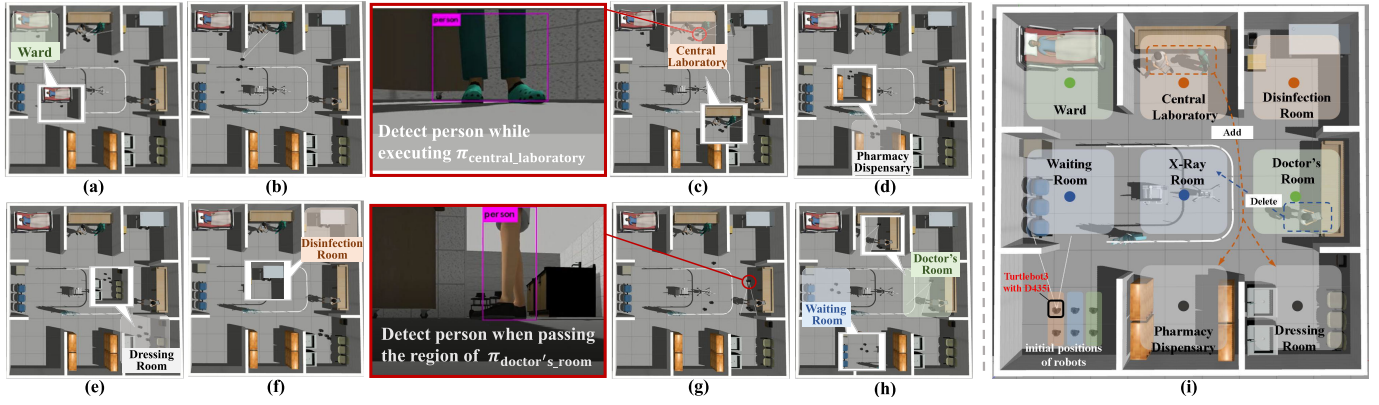


Fig. 6. Results of experimental simulation in Gazebo. (a) Execution of π_{ward} . (b) and (c) Execution of $\pi_{central_laboratory}$. The robot with perception module detects the landmark “person” in the region of $\pi_{central_laboratory}$ and the local task φ_{new} is added. (d) and (e) Executions of the newly added tasks $\pi_{pharmacy_dispensary}$ and $\pi_{dressing_room}$. (f) Execution of $\pi_{disinfection_room}$. (g) Robot with perception module detects the landmark “person” in the region of $\pi_{doctor's_room}$ and the task π_{X-Ray_room} is removed. (h) Executions of $\pi_{doctor's_room}$ and $\pi_{waiting_room}$. (i) Schematic of a hospital scenario.

TABLE V

COMPUTATIONAL TIME FOR DIFFERENT NUMBERS OF DELETED ATOMIC PROPOSITIONS WITH DIFFERENT RESOURCES

Deleted Atomic Proposition	$num_{B,s}$	Time(s) (inherent)	Time(s) (time – dependent)	Time(s) (task – dependent)	Time(s) (All)
π_2	8	0.2457	0.2431	0.2426	0.2374
π_2, π_3	7	0.2204	0.2283	0.2166	0.2262
$\pi_2 - \pi_4$	6	0.2121	0.2116	0.2063	0.2006
$\pi_2 - \pi_5$	5	0.1875	0.1944	0.1919	0.1893
$\pi_2 - \pi_6$	4	0.1730	0.1760	0.1679	0.1761
$\pi_2 - \pi_7$	3	0.1590	0.1649	0.1621	0.1550
$\pi_2 - \pi_8$	2	0.1550	0.1439	0.1435	0.1485

TABLE VI

COMPUTATIONAL TIME FOR DIFFERENT NUMBERS OF DELETED ATOMIC PROPOSITIONS UNDER DIFFERENT TYPES OF INTER-TASK CONSTRAINTS

Deleted Atomic Proposition	$num_{B,s}$	Time(s) (unrelated)	Time(s) (compatible)	Time(s) (exclusive)	Time(s) (All)
π_2	8	0.2457	0.2256	0.3299	0.3012
π_2, π_3	7	0.2204	0.2008	0.3099	0.2933
$\pi_2 - \pi_4$	6	0.2121	0.1960	0.2848	0.2824
$\pi_2 - \pi_5$	5	0.1875	0.1686	0.2748	0.2512
$\pi_2 - \pi_6$	4	0.1730	0.1522	0.2547	0.2437
$\pi_2 - \pi_7$	3	0.1590	0.1524	0.2353	0.2306
$\pi_2 - \pi_8$	2	0.1550	0.1419	0.1935	0.1801

TABLE VII

COMPARISON OF COMPUTATION TIME WITH MILP

$ S $	Number of Robots	Number of Resource Categories	MILP (s)	Ours (s)
4	12 (4)	3	1.7857	0.0636
8	12 (4)	3	7.6586	0.1354
16	12 (4)	3	24.0690	0.2904
16	12 (12)	1	2.1289	0.3143
16	12 (6)	2	3.3869	0.2902
16	12 (3)	4	8.8630	0.3170
16	60 (20)	3	17.2649	0.3043
16	300 (100)	3	53.5550	0.4184
16	600 (200)	3	59.0106	0.5592

According to Tables V and VI, our method allows for promptly reactive planning due to the removal of atomic propositions. In these tables, $num_{B,s}$ represents the number of automaton states for the new formula, and the computation time decreases as

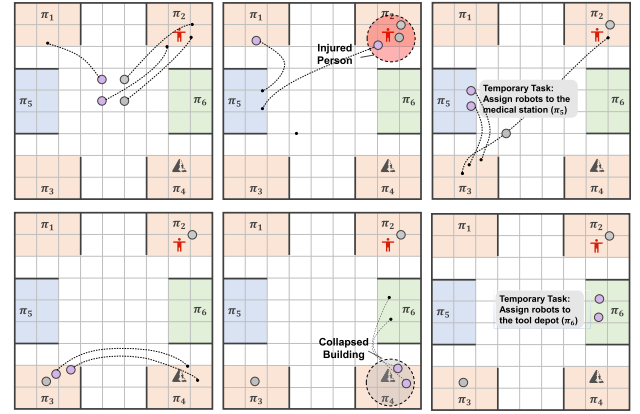


Fig. 7. Schematic of the experimental results of the multirobot search and rescue scenario.

$num_{B,s}$ decreases. Consistent with the conclusions drawn from the previous two simulation experiments, the computation time increases with the addition of exclusive tasks due to the increased complexity.

4) *Numerical Comparison Experiments:* Since MILP is a SOTA method for task allocation and planning—adopted by approaches such as cLTL and CaTL—our method is compared against MILP in terms of computational efficiency for task allocation, as shown in Table VII.

The experiments are conducted under various configurations by varying the number of automaton states ($|S|$), the number of robots, and the number of resource categories under the fixed environment. The global task is $\phi = (\diamond\pi_1 \wedge \diamond\pi_2 \wedge \diamond\pi_3 \wedge \diamond\pi_4)^{R,b}$, where $b = 0$. Based on the results, our method shows stable performance under increasing resource complexity and robot team sizes, whereas MILP suffers from rapidly growing computation time, which demonstrates the scalability and efficiency of our proposed method.

The above numerical experiments demonstrate the following features.

- 1) Our method efficiently generates feasible solutions in a short time and reactively plans new solutions in response to changes in task requirements.

TABLE VIII
TASK ALLOCATION AND CORRESPONDING VALUES OF RESOURCES (I.E., CURRENT CAPABILITIES $\tilde{R}$) POSSESSED BY ROBOTS

Atomic Proposition	Task Allocation (Z)	a_1	a_2	a_3	a_4	a_5	a_6
initial state	[0, 0, 0, 0, 0, 0]	(6, 6, 80)	(6, 6, 80)	(5, 7, 60)	(5, 7, 60)	(7, 5, 60)	(7, 5, 60)
π_{ward}	[0, 0, 0, 1, 0, 1]	(6, 6, 79.8760)	(6, 6, 79.8760)	(5, 7, 59.8760)	(5, 8, 59.8760)	(7, 5, 59.8760)	(7, 6, 59.8760)
$\pi_{central_laboratory}$	[1, 0, 1, 0, 1, 0]	(6, 6, 279.7520)	(6, 6, 79.7520)	(5, 7, 259.7520)	(5, 8, 59.7520)	(7, 5, 259.7520)	(7, 6, 59.7520)
$\pi_{pharmacy_dispensary}$	[0, 0, 1, 1, 1, 1]	(6, 6, 279.6923)	(6, 6, 79.6923)	(5, 8, 359.6923)	(5, 9, 159.6923)	(7, 6, 359.6923)	(7, 7, 159.6923)
$\pi_{dressing_room}$	[0, 0, 1, 1, 1, 1]	(6, 6, 279.5726)	(6, 6, 79.5726)	(5, 8, 359.5726)	(5, 9, 159.5726)	(7, 6, 359.5726)	(7, 7, 159.5726)
$\pi_{disinfection_room}$	[1, 1, 0, 0, 0, 0]	(6, 5, 279.5593)	(6, 5, 79.5593)	(5, 8, 359.5593)	(5, 9, 159.5593)	(7, 6, 359.5593)	(7, 7, 159.5593)
$\pi_{waiting_room}$	[0, 1, 0, 0, 1, 1]	(6, 5, 279.2277)	(6, 5, 79.2277)	(5, 8, 359.2277)	(5, 9, 159.2277)	(7, 6, 359.2277)	(7, 7, 159.2277)
$\pi_{doctor's_room}$	[1, 0, 1, 1, 0, 0]	(6, 5, 279.2277)	(6, 5, 79.2277)	(5, 8, 359.2277)	(5, 9, 159.2277)	(7, 6, 359.2277)	(7, 7, 159.2277)

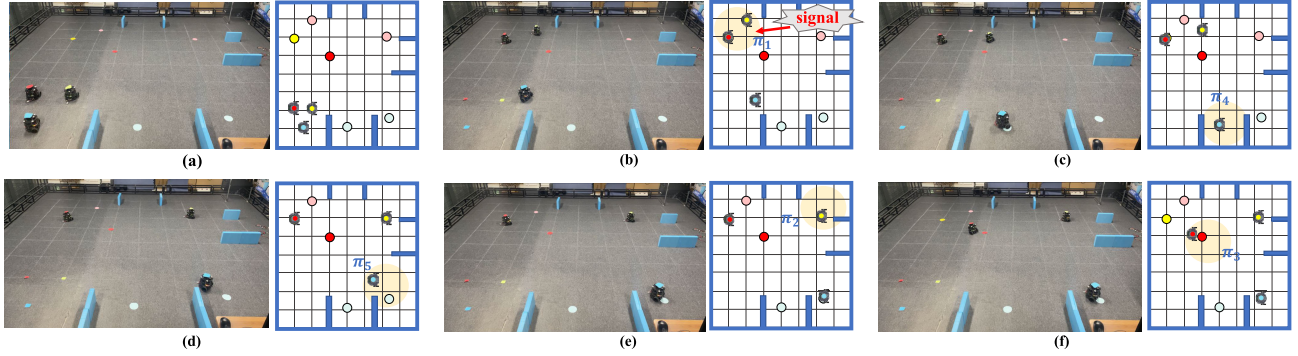


Fig. 8. Physical experiment with three turtlebots, labeled as a_1 (red), a_2 (yellow), a_3 (blue). (a) Initial state. (b) Turtlebots receive a signal to add a new task when they execute π_1 . (c) Execution of π_4 . (d) Execution of π_5 . (e) Execution of π_2 . (f) Execution of π_3 .

- 2) It is capable of handling tasks that feature multiple types of resources as well as those with different constraints on inter-task relationships.
- 3) The method shows enhanced scalability, accommodating an expanding number of robots and adeptly addressing tasks with a different number of automaton states and varied constraints.

B. Simulation Experiment

1) *Case 1:* Consider a hospital scenario as illustrated in Fig. 6(i). A set of 6 Turtlebots is employed to perform the tasks in Examples 1 and 2. Assume that one of them is equipped with a Realsense D435 camera for visual perception. The initial resources possessed by each robot are detailed in the first row of Table VIII. The LC is a nonstorage capability with an inherent corresponding resource. The amount of disinfectant each robot possesses varies with tasks according to the function $v_{DC} = v_{DC}^0 + k_1 I(\pi_{ward}) + k_2 I(\pi_{disinfection_room}) + k_3 I(\pi_{pharmacy_dispensary})$, where $I(\cdot)$ is the indicator function, and $k_1, k_2, k_3 \in \mathbb{R}$. This implies that certain equipment in the task regions of the ward, disinfection room, and pharmacy dispensary can replenish or consume the volume of disinfectant in the robot. The virus testing activity (i.e., the quantity of reagent in the robot to test for viruses) is both task-dependent and time-dependent, changing according to the formula $v_{VC} = v_{VC}^0 + k_4 I(\pi_{central_laboratory}) + k_5 I(\pi_{pharmacy_dispensary}) + f_{VC}(t)$, where $k_4, k_5 \in \mathbb{R}$ and $f_{VC}(t)$ a time-dependent function. This means that equipment in the laboratory and pharmacy dispensary can replenish or consume the quantity of reagent in the robot, while the quantity of reagent decreases over time. In this scenario, we set $k_1 = k_3 = 1$, $k_2 = -1$, $k_4 = 200$, $k_5 = 100$ and $f_{VC}(t) = -t/500$.

2) *Case 2:* The simulation results are shown in Fig. 6. Initially, the robot visits the ward to complete π_{ward} [see Fig. 6(a)]. During the process of performing $\pi_{central_laboratory}$, the robot detects the landmark “person,” prompting the robot to go to the pharmacy dispensary and dressing room to complete $\pi_{pharmacy_dispensary}$ and $\pi_{dressing_room}$ instead of heading to the disinfection room, as shown in Fig. 6(b)–(e). After completing ϕ_{new} , the robot system begins to execute the subsequent tasks, including visiting the disinfection room [see Fig. 6(f)]. While passing by the doctor’s room, the robot detects the landmark “person” [see Fig. 6(g)] and thus decides not to perform π_{X_ray} . In the subsequent process [see Fig. 6(h)], it can be observed that the robot system does not execute π_{X_ray} .

The robots selected for each task, and their corresponding current resource values, are shown in Table VIII. The bold texts in this table highlight that the robots are assigned to perform the task represented by the row. It is observable that the current resource values vary reasonably with tasks or over time, consistent with their properties as described earlier. Notably, the time-dependent resources possessed by the assigned robots remain constant during the execution of $\pi_{doctor's_room}$, as the robots arrive directly at the task region to shorten the overall execution time. Therefore, after performing $\pi_{disinfection_room}$, the robots of a_2 , a_5 , and a_6 go directly to the waiting room, while the robots of a_1 , a_3 , and a_4 go directly to the doctor’s room.

To further validate the proposed framework, we simulate a multirobot search-and-rescue task, where $\pi_1 - \pi_4$ are search regions, π_5 is the medical station, and π_6 is the tool depot. Four robots, divided into two categories, are equipped with air filtration (time-dependent resource), medicine transportation (task-dependent resource), and detection (nonstorage capability). When a robot detects an injured person, the system must

assign robots to the medical station to notify the doctors. Similarly, when a collapsed building is found, the robots need to go to the tool depot to inform the rescue personnel to fetch tools. In this experiment, we assume that the robots discover an injured person while executing task π_2 , and a collapsed building while patrolling π_4 . Assuming unrelated tasks and sufficient resources possessed by robots, the results, as shown in Fig. 7 demonstrate the efficiency of our proposed method in handling dynamically triggered tasks.

C. Physical Experiment

In this section, three turtlebots (i.e., a_1, a_2, a_3) are employed to complete the following task $\phi = \varphi^{R_{ini}+1}$, where $\varphi = \square\Diamond\pi_1 \wedge \square\Diamond\pi_2 \wedge \square\Diamond\pi_3$. Assume that the resources required for tasks and capabilities possessed by each robot are the same as in case 1. Suppose that, when a_1 executes π_1 , the system receives a command to immediately execute a new local task $\phi_L = \varphi_L^{R_L-1}$ with $\varphi_L = \square\Diamond\pi_4 \wedge \square\Diamond\pi_5$. The results are shown in Fig. 8, which shows that a_3 immediately starts to execute ϕ_L . Since a_2 assigned to π_2 does not need to complete other tasks in the subsequent task, it goes directly to the area where π_2 is located. Experimental results show that our method effectively handles multiconstraint tasks and can reactively plan feasible solutions to address the temporal changes in local tasks.

V. CONCLUSION

This work presents a formal reactive task allocation framework for addressing tasks in heterogeneous multirobot systems with multiple constraints in environments with known geometry and unknown semantic landmarks. Our ongoing research aims to address complex task allocation challenges for heterogeneous multirobot systems in dynamic environments with unknown geometry. Additionally, we will investigate how environmental resources and constraints related to the robots' capabilities can impact the final task allocation scheme. Furthermore, we seek to improve the efficiency of solving hierarchical forms of temporal logic.

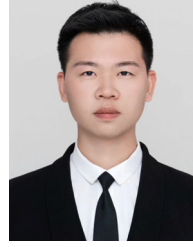
ACKNOWLEDGMENT

The AI-driven experiments, simulations, and model training were performed on the robotic AI-Scientist platform of the Chinese Academy of Sciences.

REFERENCES

- [1] S. C. Mohamed, A. Fung, and G. Nejat, "A multirobot person search system for finding multiple dynamic users in human-centered environments," *IEEE Trans. Cybern.*, vol. 53, no. 1, pp. 628–640, Jan. 2023.
- [2] Z. Lian, P. Shi, C.-C. Lim, and X. Yuan, "Fuzzy-model-based lateral control for networked autonomous vehicle systems under hybrid cyber-attacks," *IEEE Trans. Cybern.*, vol. 53, no. 4, pp. 2600–2609, Apr. 2023.
- [3] R. Chai et al., "A two phases multiobjective trajectory optimization scheme for multi-UGVs in the sight of the first aid scenario," *IEEE Trans. Cybern.*, vol. 54, no. 9, pp. 5078–5091, Sep. 2024.
- [4] G. Gao, Y. Mei, B. Xin, Y.-H. Jia, and W. N. Browne, "Automated coordination strategy design using genetic programming for dynamic multipoint dynamic aggregation," *IEEE Trans. Cybern.*, vol. 52, no. 12, pp. 13521–13535, Dec. 2022.
- [5] J. Qin, Q. Ma, H. Gao, Y. Shi, and Y. Kang, "On group synchronization for interacting clusters of heterogeneous systems," *IEEE Trans. Cybern.*, vol. 47, no. 12, pp. 4122–4133, Dec. 2017.
- [6] Q. Liu, K. Huang, H. Liu, R. Wang, and G. Cheng, "Multi-domain collaborative task allocation and conflict resolution in unmanned systems under complex constraints," *Unmanned Syst.*, vol. 13, no. 2, pp. 579–589, Mar. 2025.
- [7] Á. Calvo and J. Capitán, "Heterogeneous multirobot task allocation for long-endurance missions in dynamic scenarios," *IEEE Trans. Robot.*, vol. 41, pp. 6494–6513, 2025.
- [8] H. Xiao, R. Cui, and D. Xu, "A sampling-based Bayesian approach for cooperative multiagent online search with resource constraints," *IEEE Trans. Cybern.*, vol. 48, no. 6, pp. 1773–1785, Jun. 2018.
- [9] M. Shahriari and M. Biglarbegian, "Toward safer navigation of heterogeneous mobile robots in distributed scheme: A novel time-to-collision-based method," *IEEE Trans. Cybern.*, vol. 52, no. 9, pp. 9302–9315, Sep. 2022.
- [10] R. Hong, J. Zhang, M. Gan, Q. Wang, B. Xin, and J. Chen, "Post-disaster multi-UAV task planning based on graph neural network decoupling," *Unmanned Syst.*, pp. 1–16, Jan. 2026.
- [11] C. Zhou, J. Yang, S. Li, and W.-H. Chen, "Temporal logic disturbance rejection control of nonlinear systems using control barrier functions," *IEEE Trans. Cybern.*, vol. 55, no. 3, pp. 1008–1017, Mar. 2025.
- [12] C. Liu, J. Cong, B. Chen, Y. Jin, and E. Zhu, "Guiding multiagent multitask reinforcement learning by a hierarchical framework with logical reward shaping," *IEEE Trans. Cybern.*, vol. 56, no. 3, pp. 1569–1582, Mar. 2026.
- [13] Y. E. Sahin, P. Nilsson, and N. Ozay, "Multirobot coordination with counting temporal logics," *IEEE Trans. Robot.*, vol. 36, no. 4, pp. 1189–1206, Aug. 2020.
- [14] K. Leahy et al., "Scalable and robust algorithms for task-based coordination from high-level specifications (ScRATChES)," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2516–2535, Aug. 2022.
- [15] L. Govoni and A. Cristofaro, "A decentralized control and task allocation framework for heterogeneous redundant multiagent systems," *IEEE Trans. Cybern.*, vol. 55, no. 12, pp. 5605–5618, Dec. 2025.
- [16] G. A. Cardona and C.-I. Vasile, "Planning for heterogeneous teams of robots with temporal logic, capability, and resource constraints," *Int. J. Robot. Res.*, vol. 43, no. 13, pp. 2089–2111, Nov. 2024.
- [17] Z. Duan, G. Xu, Z. Li, and M. Ji, "Heterogeneous multiagent task allocation via cooperative exchange strategies for equilibrium-improving: A potential game framework," *IEEE Trans. Cybern.*, vol. 56, no. 2, pp. 954–967, Feb. 2026.
- [18] P. Chen et al., "Multiobjective task allocation for electric harvesting robots: A hierarchical route reconstruction approach," *IEEE Trans. Cybern.*, vol. 56, no. 3, pp. 1583–1596, Mar. 2026.
- [19] Q. Lu and Q.-L. Han, "Mobile robot networks for environmental monitoring: A cooperative receding horizon temporal logic control approach," *IEEE Trans. Cybern.*, vol. 49, no. 2, pp. 698–711, Feb. 2019.
- [20] M. Cai, S. Xiao, Z. Li, and Z. Kan, "Optimal probabilistic motion planning with potential infeasible LTL constraints," *IEEE Trans. Autom. Control*, vol. 68, no. 1, pp. 301–316, Jan. 2023.
- [21] Z. Liu, M. Guo, and Z. Li, "Time minimization and online synchronization for multi-agent systems under collaborative temporal logic tasks," *Automatica*, vol. 159, Jan. 2024, Art. no. 111377.
- [22] A. Lavaei, S. Soudjani, A. Abate, and M. Zamani, "Automated verification and synthesis of stochastic hybrid systems: A survey," *Automatica*, vol. 146, Dec. 2022, Art. no. 110617.
- [23] Z. Xu and A. A. Julius, "Census signal temporal logic inference for multiagent group behavior analysis," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 1, pp. 264–277, Jan. 2018.
- [24] X. Luo and C. Liu, "Simultaneous task allocation and planning for multirobots under hierarchical temporal logic specifications," *IEEE Trans. Robot.*, vol. 41, pp. 5040–5059, 2025.
- [25] Y. Huang, N. Xu, M. Guo, J. Li, and L. Shen, "Distributed and reactive controller synthesis for multi-agent systems under finite horizon temporal logic tasks," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 13485–13500, 2025.
- [26] L. Li, Z. Chen, H. Wang, and Z. Kan, "Fast task allocation of heterogeneous robots with temporal logic and inter-task constraints," *IEEE Robot. Autom. Lett.*, vol. 8, no. 8, pp. 4991–4998, Aug. 2023.
- [27] L. Li, Z. Chen, H. Wang, and Z. Kan, "Task allocation of heterogeneous robots under temporal logic specifications with inter-task constraints and variable capabilities," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 14030–14047, 2025.
- [28] Y. Kantaros and M. M. Zavlanos, "Sampling-based optimal control synthesis for multirobot systems under global temporal tasks," *IEEE Trans. Autom. Control*, vol. 64, no. 5, pp. 1916–1931, May 2019.
- [29] Y. Kantaros, M. Guo, and M. M. Zavlanos, "Temporal logic task planning and intermittent connectivity control of mobile robot networks," *IEEE Trans. Autom. Control*, vol. 64, no. 10, pp. 4105–4120, Oct. 2019.

- [30] D. Xu, K. Ye, Z. Zheng, T. Zhou, G. G. Yen, and M. Jiang, "An efficient dynamic resource allocation framework for evolutionary bilevel optimization," *IEEE Trans. Cybern.*, vol. 55, no. 2, pp. 726–739, Feb. 2025.
- [31] D. R. Martin, B. A. Butler, S. Nivison, M. Egerstedt, M. A. Al Faruque, and P. Khargonekar, "Collaborative task allocation for heterogeneous multi-robot systems through iterative clustering," *IEEE Robot. Autom. Lett.*, vol. 11, no. 1, pp. 33–40, Jan. 2026.
- [32] A. Bono, G. Fedeles, and G. Franze, "A swarm-based distributed model predictive control scheme for autonomous vehicle formations in uncertain environments," *IEEE Trans. Cybern.*, vol. 52, no. 9, pp. 8876–8886, Sep. 2022.
- [33] M. Lahijanian, M. R. Maly, D. Fried, L. E. Kavraki, H. Kress-Gazit, and M. Y. Vardi, "Iterative temporal planning in uncertain environments with partial satisfaction guarantees," *IEEE Trans. Robot.*, vol. 32, no. 3, pp. 583–599, Jun. 2016.
- [34] V. Vasilopoulos, W. Vega-Brown, O. Arslan, N. Roy, and D. E. Koditschek, "Sensor-based reactive symbolic planning in partially known environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 5683–5690.
- [35] Y. Kantaros, S. Kalluraya, Q. Jin, and G. J. Pappas, "Perception-based temporal logic planning in uncertain semantic maps," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2536–2556, Aug. 2022.
- [36] L. Li, Z. Chen, and Z. Kan, "Environment-driven and LLM-guided multi-robot task inference and allocation under temporal logic specifications," *IEEE Trans. Autom. Sci. Eng.*, vol. 23, pp. 2925–2940, 2026.
- [37] S. Mahmoudzadeh, D. M. W. Powers, and A. Atyabi, "UUV's hierarchical DE-based motion planning in a semi dynamic underwater wireless sensor network," *IEEE Trans. Cybern.*, vol. 49, no. 8, pp. 2992–3005, Aug. 2019.
- [38] Z. Zhou, D. J. Lee, Y. Yoshinaga, S. Balakirsky, D. Guo, and Y. Zhao, "Reactive task allocation and planning for quadrupedal and wheeled robot teaming," in *Proc. IEEE 18th Int. Conf. Autom. Sci. Eng. (CASE)*, Aug. 2022, pp. 2110–2117.
- [39] Z. Chen and Z. Kan, "Real-time reactive task allocation and planning of large heterogeneous multi-robot systems with temporal logic specifications," *Int. J. Robot. Res.*, vol. 44, no. 4, pp. 640–664, Apr. 2025.
- [40] Z. Chen, Z. Zhou, S. Wang, J. Li, and Z. Kan, "Fast temporal logic mission planning of multiple robots: A planning decision tree approach," *IEEE Robot. Autom. Lett.*, vol. 9, no. 7, pp. 6146–6153, Jul. 2024.
- [41] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.
- [42] R. Martins, D. Bersan, M. F. M. Campos, and E. R. Nascimento, "Extending maps with semantic and contextual object information for robot navigation: A learning-based framework using visual and depth cues," *J. Intell. Robot. Syst.*, vol. 99, nos. 3–4, pp. 555–569, Sep. 2020.
- [43] C. Baier and J.-P. Katoen, *Principles of Model Checking*. Cambridge, MA, USA: MIT Press, 2008.
- [44] P. Gastin and D. Oddoux, "Fast LTL to Büchi automata translation," in *Proc. Comput. Aided Verification, 13th Int. Conf.*, Paris, France, 2001, pp. 53–65.



Zhangli Zhou received the B.E. degree from Wuhan University of Technology, Wuhan, China, in 2016, and the Ph.D. degree from the University of Science and Technology of China, Hefei, China, in 2024.

He is currently a Post-Doctoral Researcher with the University of Science and Technology of China. His research interests include robotics and reactive task and motion planning.



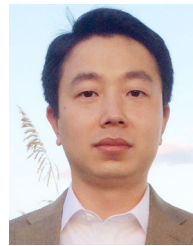
Ziyang Chen received the B.E. degree from Beihang University, Beijing, China, in 2020, and the Ph.D. degree in control science and engineering from the University of Science and Technology of China, Hefei, China, in 2025.

He is currently working with Momena, Shanghai, China. His current research interests include robotics and autonomous vehicles.



Hao Wang received the B.S. degree in mechanical engineering from China University of Mining and Technology, Xuzhou, China, in 2017, and the Ph.D. degree in control science and engineering from the University of Science and Technology of China, Hefei, China, in 2023.

He is currently working with DJI, Shenzhen, China. His current research interests include motion planning in robotics, manipulation skill learning, and deep reinforcement learning.



Zhen Kan (Senior Member, IEEE) received the Ph.D. degree from the Department of Mechanical and Aerospace Engineering, University of Florida, Gainesville, FL, USA, in 2011.

He is currently a Professor at the Department of Automation, University of Science and Technology of China, Hefei, China. His research interests include control theory, formal methods, and robotics.

Dr. Kan serves as an Associate Editor for *International Journal of Robotics Research*, *IEEE TRANSACTIONS ON AUTOMATIC CONTROL*, and

IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS.



Lin Li (Graduate Student Member, IEEE) received the B.E. degree in mechanical engineering and automation from Hefei University of Technology, Hefei, China, in 2021. She is currently pursuing the Ph.D. degree in control engineering with the University of Science and Technology of China, Hefei.

Her current research interests include multirobot systems and formal methods.



Jiahui Qin (Senior Member, IEEE) received the first Ph.D. degree in control science and engineering from Harbin Institute of Technology, Harbin, China, in 2012, and the second Ph.D. degree in systems and control from Australian National University, Canberra, ACT, USA, in 2014.

He is currently a Professor with the Department of Automation, University of Science and Technology of China, Hefei, China. His current research interests include networked control systems, autonomous intelligent systems, and human-robot interaction.